



**UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE
COLOMBIA**

**FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA DE SISTEMAS**

YUNG ACCOUNTANT
Sistema para la gestión de finanzas personales

CAMILO ANTONIO MERCHAN SANTOS

Arquitectura de Sistemas - Proyecto Final

Ingeniería de Sistemas

03 de Junio de 2026

Docente:

Jorge Armando Pérez Abril

Correo electrónico:

camilo.merchan02@uptc.edu.co

YUNG ACCOUNTANT

Sistema para la gestión de finanzas personales

CAMILO ANTONIO MERCHAN SANTOS

Código: 202220095

Organización: Universidad Pedagógica y Tecnológica de Colombia (UPTC)
Materia: Arquitectura de Sistemas
Correo: camilo.merchan02@uptc.edu.co
Fecha: 03 de Junio de 2026

Contents

1	Introducción	4
2	Descripción del Problema	4
2.1	Antecedentes	4
2.2	Necesidades Identificadas	4
2.3	Causas del Problema	5
2.4	Formulación de la Pregunta	5
3	Justificación	5
3.1	Propósitos del Sistema	5
3.2	Partes Interesadas (Stakeholders)	5
4	Objetivos	6
4.1	Objetivo General	6
4.2	Objetivos Específicos	6
4.3	Diagrama Entidad-Relación	6
4.4	Características Especiales	7
4.4.1	pgvector para Búsqueda Semántica	7
4.4.2	pg-trgm para Búsqueda de Usuarios	7
4.4.3	Funciones PL/pgSQL Optimizadas	7
4.4.4	Triggers Automáticos	8
4.4.5	Índices Estratégicos	8
5	Modelo 4+1 de Arquitectura de Software	8
5.1	Vista Lógica	8
5.1.1	Descripción	8
5.2	Vista de Procesos	9
5.2.1	Descripción General	9
5.2.2	Proceso 1: Registro de Usuario	10
5.2.3	Proceso 2: Inicio de Sesión	10
5.2.4	Proceso 3: Crear Post con Embedding Semántico	11
5.2.5	Proceso 4: Registrar Pago de Deuda	11
5.2.6	Proceso 5: Eventos Asíncronos con Kafka	11
5.2.7	Proceso 6: Verificación de Token (Middleware)	12
5.3	Vista de Desarrollo	13
5.3.1	Descripción	13
5.4	Vista Física	13
5.4.1	Descripción	13
5.5	Escenarios (+1)	14
5.5.1	Descripción General	14
5.5.2	Escenario 1: Registro de Usuario	15
5.5.3	Escenario 2: Inicio de Sesión	15
5.5.4	Escenario 3: Crear Post con Embedding Semántico	16
5.5.5	Escenario 4: Registrar Pago de Deuda	17
5.5.6	Escenario 5: Feed Personalizado	18
5.5.7	Escenario 6: Búsqueda de Usuarios	19
6	Modelo Entidad-Relación (MER)	19
6.1	Descripción General	19
6.2	Entidades Principales	19
6.2.1	User (Usuarios)	19
6.2.2	Category (Categorías)	19
6.2.3	Wallet (Carteras)	20
6.2.4	Transaction (Transacciones)	20
6.2.5	Goal (Metas)	20
6.2.6	Debt (Deudas)	20
6.2.7	Habit (Hábitos)	20

6.2.8	Post (Publicaciones)	20
6.2.9	Comment (Comentarios)	20
6.3	Relaciones Clave	20
7	Microservicios de Infraestructura	21
7.1	Servicio de Base de Datos (PostgreSQL + pgvector)	21
7.1.1	Descripción General	21
7.1.2	¿Qué hace?	21
7.1.3	¿Cómo lo hace?	21
7.1.4	Integración	22
7.1.5	Justificación	22
7.1.6	Alcance	22
7.1.7	Configuración de Entorno	22
7.2	Servicio de Caché (Redis)	22
7.2.1	Descripción General	22
7.2.2	¿Qué hace?	22
7.2.3	¿Cómo lo hace?	23
7.2.4	Integración	23
7.2.5	Justificación	23
7.2.6	Alcance	23
7.2.7	Configuración de Entorno	23
7.3	Servicio de Autenticación y Autorización (Keycloak)	23
7.3.1	Descripción General	23
7.3.2	¿Qué hace?	23
7.3.3	¿Cómo lo hace?	24
7.3.4	Integración	24
7.3.5	Justificación	24
7.3.6	Alcance	24
7.3.7	Configuración de Entorno	24
7.4	Servicio de Mensajería y Eventos (Kafka)	24
7.4.1	Descripción General	24
7.4.2	¿Qué hace?	24
7.4.3	¿Cómo lo hace?	25
7.4.4	Integración	25
7.4.5	Justificación	25
7.4.6	Alcance	25
7.4.7	Configuración de Entorno	25
7.5	Servicio de Embeddings (ONNX Runtime)	25
7.5.1	Descripción General	25
7.5.2	¿Qué hace?	25
7.5.3	¿Cómo lo hace?	25
7.5.4	Integración	26
7.5.5	Justificación	26
7.5.6	Alcance	26
7.5.7	Configuración de Entorno	26
8	Microservicios de Negocio	26
8.1	Servicio de Autenticación (auth)	26
8.1.1	Descripción General	26
8.1.2	Endpoints de la API REST	26
8.2	Servicio de Categorías (category)	28
8.2.1	Descripción General	28
8.2.2	Endpoints de la API REST	29
8.3	Servicio de Comunidad (community)	30
8.3.1	Descripción General	30
8.3.2	Endpoints de la API REST	30
8.4	Servicio de Deudas (debt)	34
8.4.1	Descripción General	34
8.4.2	Endpoints de la API REST	34

8.5	Servicio de Metas (goal)	36
8.5.1	Descripción General	36
8.5.2	Endpoints de la API REST	36
8.6	Servicio de Hábitos (habit)	37
8.6.1	Descripción General	37
8.6.2	Endpoints de la API REST	37
8.7	Servicio de Simulación (simulation)	38
8.7.1	Descripción General	38
8.7.2	Endpoints de la API REST	38
8.8	Servicio de Transacciones (transaction)	39
8.8.1	Descripción General	39
8.8.2	Endpoints de la API REST	40
8.9	Servicio de Carteras (wallet)	41
8.9.1	Descripción General	41
8.9.2	Endpoints de la API REST	41
8.10	Frontend (React + TypeScript)	42
8.10.1	Descripción General	42
8.10.2	Arquitectura del Frontend	42
8.10.3	Endpoints del Frontend (Rutas)	43
8.10.4	Flujo de Autenticación	43
8.10.5	Flujo de Búsqueda (Navbar)	44
8.10.6	Optimistic Updates (Ejemplo: Likes)	44
8.10.7	Tecnologías y Librerías	44
8.10.8	Configuración de Entorno (Frontend)	44
8.10.9	Características Destacadas	44

1 Introducción

El sistema **Yung Accountant** es una plataforma de gestión de finanzas personales diseñada específicamente para jóvenes (denominados coloquialmente “*yung niggas*” en el documento original). La aplicación permite a los usuarios registrar transacciones, establecer metas de ahorro, gestionar deudas, seguir hábitos financieros, participar en una comunidad de apoyo, y visualizar análisis de su comportamiento financiero.

La arquitectura del sistema está compuesta por múltiples microservicios desarrollados en C++ (con Boost.Beast/Asio), un frontend en React 19 con TypeScript y TailwindCSS, y una infraestructura desplegada en Oracle Cloud Infrastructure (OCI). Los microservicios se comunican de forma síncrona mediante HTTP/REST a través de un API Gateway (Nginx), y de forma asíncrona mediante eventos a través de Kafka. La autenticación y autorización se delega a Keycloak (OpenID Connect/OAuth2), y los datos se almacenan en PostgreSQL 16 con la extensión pgvector para búsqueda semántica, con una capa de caché distribuida en Redis.

El presente documento describe la arquitectura tecnológica completa del sistema, incluyendo el modelo entidad-relación (MER), las cinco vistas del modelo 4+1 (lógica, procesos, desarrollo, física y escenarios), la documentación de los microservicios (infraestructura y negocio), y los diagramas de secuencia que ilustran los flujos principales.

2 Descripción del Problema

2.1 Antecedentes

El manejo del dinero es un proceso más complejo de lo que comúnmente se cree. El dinero gastado hoy tiende a ser olvidado mañana, y los gastos se acumulan generando pérdidas que podrían destinarse a otros propósitos como ahorros, inversiones o adquisición de equipos. La distribución del dinero mejora significativamente cuando las variables e información financiera son bien conocidas.

Adicionalmente, surge el problema de los movimientos de dinero a corto y largo plazo. Sin un registro adecuado, resulta casi imposible analizar tendencias, predecir comportamientos y calcular variables de interés como costos semanales, mensuales o anuales.

2.2 Necesidades Identificadas

El análisis del problema revela las siguientes necesidades:

- **Conciencia financiera:** Los jóvenes gastan dinero diariamente en actividades superfluas (drogas, alcohol, juegos, apuestas) sin preocuparse por el daño a su bolsillo. Cuando se dan cuenta de que el dinero se ha ido y no pueden costear objetivos importantes, caen en un ciclo vicioso.
- **Gestión financiera:** Después de abandonar malos hábitos, es necesario aprender a gestionar los ingresos, controlar gastos necesarios (alimentos, facturas) y analizar cómo se comportan los costos a lo largo del tiempo.
- **Metas financieras:** La falta de enfoque y disciplina es un problema psicológico común en los jóvenes. Establecer metas con variables de tiempo y contexto (universidad, comida, amigos, familia) permite motivarse hacia objetivos tangibles.
- **Almacenamiento financiero:** Es necesario investigar y utilizar entidades bancarias que ofrezcan tarjetas de débito o crédito, adaptando las necesidades del usuario a los diferentes productos financieros disponibles.
- **Control de compras:** Las compras diarias requieren un registro estricto, ya que pueden causar daños significativos si no se controlan adecuadamente.
- **Ahorro:** Las personas tienden a gastar el pago del trabajo inmediatamente y luego buscan préstamos. El ahorro es crítico y prioritario para alcanzar metas.
- **Control externo:** Factores externos como el gobierno y las empresas controlan los precios, pero el usuario puede utilizar esta información para mejorar sus decisiones financieras.
- **Comunidad:** La población joven carece de conocimientos en finanzas. Un espacio de socialización e intercambio de experiencias es importante para estimular la intuición, la resolución de problemas y la creación de nuevas ideas.

2.3 Causas del Problema

- **Falta de metas financieras:** Sin motivación ni interés en obtener algo, las personas no planifican el uso de su dinero.
- **Bajo control financiero:** La escuela no enseña finanzas; las personas aprenden cometiendo errores, lo que lleva a malas decisiones.
- **Falta de análisis:** No se analizan tendencias, ni se crean estadísticas ni patrones; no se ven más allá de los números.
- **Desinformación:** Los jóvenes rechazan los préstamos o créditos sin investigar los beneficios y riesgos.
- **Débil sentido de ahorro:** Es fácil gastar dinero sin darse cuenta de que no solo se gasta dinero, sino también tiempo y salud.
- **Descentralización:** No existe un espacio único donde la conversación principal sea la gestión financiera.

2.4 Formulación de la Pregunta

¿Necesitan los jóvenes, amas de casa y trabajadores un sistema financiero para gestionar cuentas, controlar decisiones, establecer metas, seguir tendencias y analizar estadísticas para sus finanzas personales?

3 Justificación

3.1 Propósitos del Sistema

El sistema Yung Accountant se justifica por los siguientes propósitos:

- **Aumentar la conciencia financiera:** El sistema ayudará a los jóvenes a gamificar su relación con el dinero, preocupándose más por sus problemas financieros. Los padres pueden enseñar a sus hijos a gestionar su propio dinero desde temprana edad.
- **Establecer metas:** Los jóvenes obtienen una gran ventaja si juegan de manera inteligente con el tiempo y el dinero. Las metas requieren tanto dinero como tiempo para lograr cosas valiosas como una bicicleta, un coche, inversiones o ahorros.
- **Gestionar almacenamiento, compras y ahorros:** Enseñar a gastar dinero es un proceso natural de experimentación. El sistema busca evitar malas decisiones en los jóvenes, controlando todos estos flujos de dinero y permitiendo analizar múltiples escenarios antes de tomar decisiones.
- **Mejorar las decisiones financieras:** Es necesario reducir las malas decisiones al mínimo, abordando todos los datos necesarios para estructurar un sistema sólido de asesoramiento financiero.
- **Análisis de datos:** Los datos construyen información; el análisis ayuda a la persona a visualizar diferentes perspectivas que normalmente no ve, permitiendo identificar oportunidades de mejora.
- **Comunidad fuerte:** Diferentes opiniones son mejores que una. Socializar metas, procesos de gestión y experiencias ayuda a llenar el conocimiento con consejos y comentarios de personas reales con problemas reales.

3.2 Partes Interesadas (Stakeholders)

- **Grupo de programación:** Encargado del desarrollo de los microservicios en C++.
- **Grupo de documentación:** Responsable de la documentación técnica y de usuario.
- **Grupo de experticia financiera:** Asesoran sobre la lógica financiera del sistema.
- **Jóvenes (target demographic):** Usuarios finales interesados en gestionar sus finanzas y metas.

- **Bancos:** Posible colaboración para implementar características directamente con entidades financieras.
- **Escuelas:** Alianzas para crear espacios de reflexión sobre decisiones financieras y motivar el uso del sistema.

4 Objetivos

4.1 Objetivo General

Crear un sistema que permita a los usuarios gestionar sus finanzas personales, cubriendo sus compras, ahorros y almacenamiento financiero, así como establecer y dar seguimiento a metas utilizando análisis de datos, gamificación y técnicas financieras, e implementar un espacio seguro para compartir ideas, hacer preguntas, publicar contenidos y comentarios.

4.2 Objetivos Específicos

- Crear documentación específica que cubra todos los problemas, incluyendo requisitos de usuarios, metodologías, recursos teóricos y encuestas para obtener la máxima información que los usuarios necesitan para el sistema.
- Documentar el diseño del software buscando herramientas, algoritmos, bases de datos, servidores, normativas, integración, mantenimiento, etc.
- Crear diagramas que muestren el flujo de la aplicación (modelo 4+1: lógica, procesos, desarrollo, física y escenarios).
- Crear mockups del sistema con técnicas de UI/UX orientadas a jóvenes.
- Codificar el sistema en software, siguiendo la metodología establecida.
- Probar el sistema con un grupo de usuarios.
- Subir el sistema a producción (Oracle Cloud Infrastructure).

4.3 Diagrama Entidad-Relación

El siguiente diagrama muestra todas las entidades, sus atributos y las relaciones entre ellas, incluyendo multiplicidades y claves primarias/foráneas.

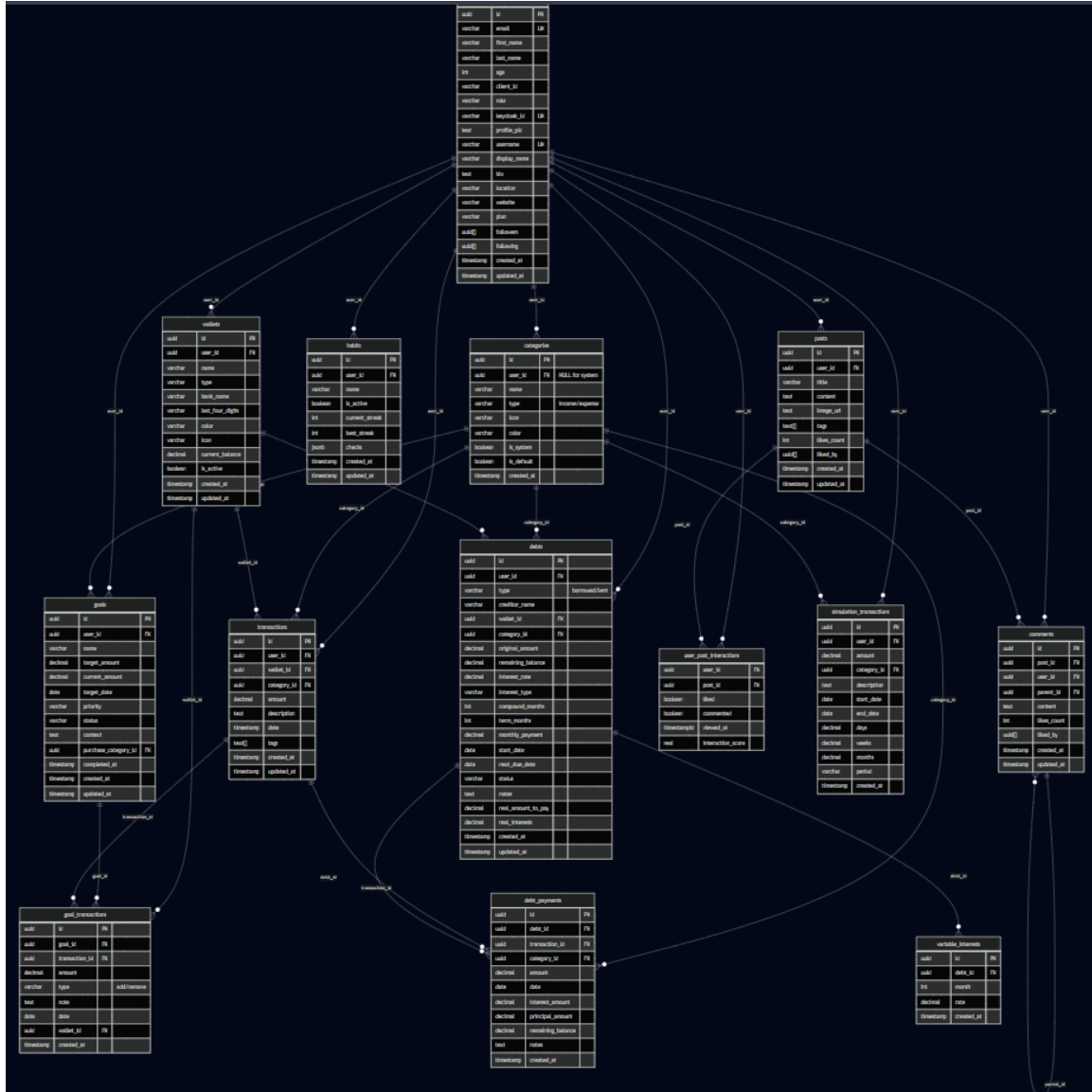


Figure 1: Modelo Entidad-Relación del sistema Yung Accountant

4.4 Características Especiales

4.4.1 pgvector para Búsqueda Semántica

La tabla `posts` incluye una columna `embedding vector(384)` que almacena los embeddings generados por el servicio ONNX Runtime. Esto permite realizar búsquedas por similitud semántica mediante el operador `<=>` (cosine distance) y la función `search_posts_semantic`.

4.4.2 pg_trgm para Búsqueda de Usuarios

La búsqueda de usuarios utiliza la extensión `pg_trgm` (trigramas) con índices GIN en los campos `username`, `display_name`, `first_name`, `last_name` y `bio`, permitiendo búsquedas eficientes con el operador `%`.

4.4.3 Funciones PL/pgSQL Optimizadas

El sistema incluye varias funciones almacenadas para optimizar operaciones complejas:

- `search_posts_optimized`: Búsqueda híbrida (full-text + trigramas)
- `get_personalized_feed`: Feed con scoring por relevancia
- `search_users_trgm`: Búsqueda de usuarios por similitud

- `get_trending_posts`: Posts populares basados en likes y comentarios

4.4.4 Triggers Automáticos

Se implementan triggers para:

- Actualizar automáticamente `updated_at` en todas las tablas
- Actualizar `search_vector` en la tabla `posts` al insertar o actualizar

4.4.5 Índices Estratégicos

Se han creado índices específicos para consultas frecuentes:

- Índices GIN para búsqueda trigramma (`pg_trgm`) en usuarios y posts
- Índice `ivfflat` para búsqueda vectorial (`pgvector`)
- Índices B-tree en claves foráneas y campos de fecha

5 Modelo 4+1 de Arquitectura de Software

El modelo de arquitectura 4+1, propuesto por Philippe Kruchten, organiza la descripción de un sistema software en cinco vistas concurrentes que abordan los intereses de los diferentes stakeholders. Este documento aplica dicho modelo al sistema Yung Accountant, una plataforma de gestión de finanzas personales orientada a jóvenes.

Las cinco vistas son:

1. **Vista Lógica:** Describe el modelo de objetos y las clases del sistema.
2. **Vista de Procesos:** Describe los aspectos de concurrencia, sincronización y flujos de trabajo.
3. **Vista de Desarrollo:** Describe la organización estática del software en su entorno de desarrollo.
4. **Vista Física:** Describe el mapeo del software al hardware y los aspectos de distribución. item **Escenarios (+1):** Ilustran la arquitectura mediante casos de uso concretos.

5.1 Vista Lógica

5.1.1 Descripción

La vista lógica representa el modelo de clases UML del sistema Yung Accountant. Muestra las entidades principales del dominio, sus atributos, métodos y las relaciones entre ellas (asociación, composición y dependencia). Esta vista es fundamental para comprender la estructura de datos del sistema y las responsabilidades de cada entidad.

Las clases principales incluyen:

- **User:** Usuario del sistema con autenticación vía Keycloak.
- **Category:** Categorías de ingresos y gastos (sistema o personalizadas).
- **Wallet:** Carteras financieras (efectivo, cuentas bancarias, tarjetas).
- **Transaction:** Transacciones financieras asociadas a wallets y categorías.
- **Goal:** Metas de ahorro con seguimiento de progreso.
- **Debt:** Préstamos y deudas con interés compuesto.
- **Habit:** Hábitos diarios con rachas (streaks).
- **Post:** Publicaciones en la comunidad.
- **Comment:** Comentarios y respuestas anidadas.

Las relaciones clave son:

- User 1 → * Category, Wallet, Transaction, Goal, Debt, Habit, Post, Comment
- Category 1 → * Transaction
- Wallet 1 → * Transaction
- Goal 1 → * GoalTransaction (composición)
- Debt 1 → * DebtPayment, * VariableInterest (composición)
- Post 1 → * Comment (composición)
- Comment 0..1 → * Comment (auto-referencia para respuestas)

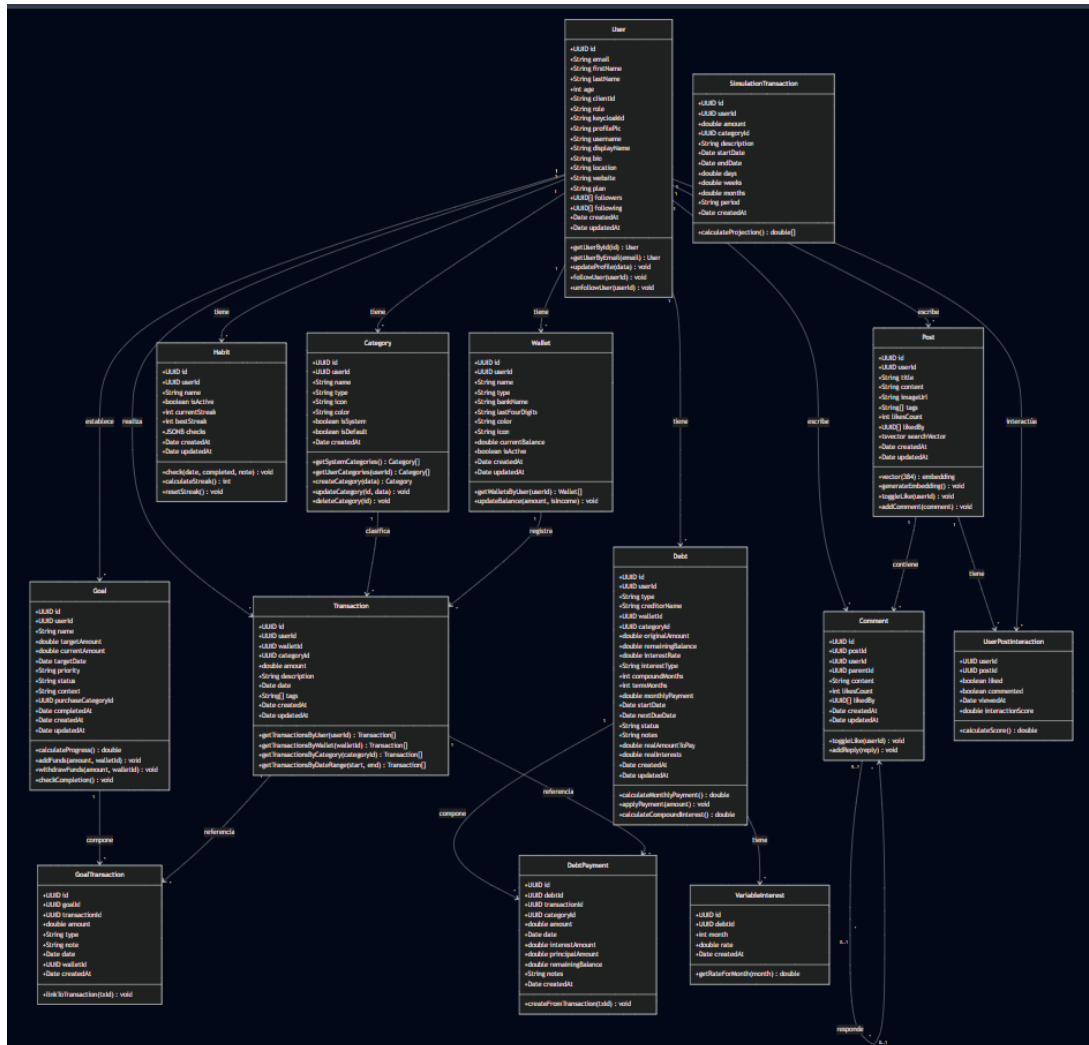


Figure 2: Diagrama de clases UML del sistema Yung Accountant

5.2 Vista de Procesos

5.2.1 Descripción General

La vista de procesos describe los flujos de trabajo, la concurrencia y la comunicación entre los componentes del sistema. Incluye diagramas de secuencia para los procesos críticos del sistema, mostrando la interacción temporal entre actores, microservicios, bases de datos y servicios externos.

5.2.2 Proceso 1: Registro de Usuario

Este diagrama muestra el flujo completo de registro de un nuevo usuario. El proceso involucra al cliente React, el API Gateway, el servicio de autenticación, PostgreSQL (verificación y creación), Keycloak (gestión de identidad) y Kafka (evento de auditoría). Incluye validación de datos, verificación de email duplicado, creación en PostgreSQL, registro en Keycloak y asignación de roles.

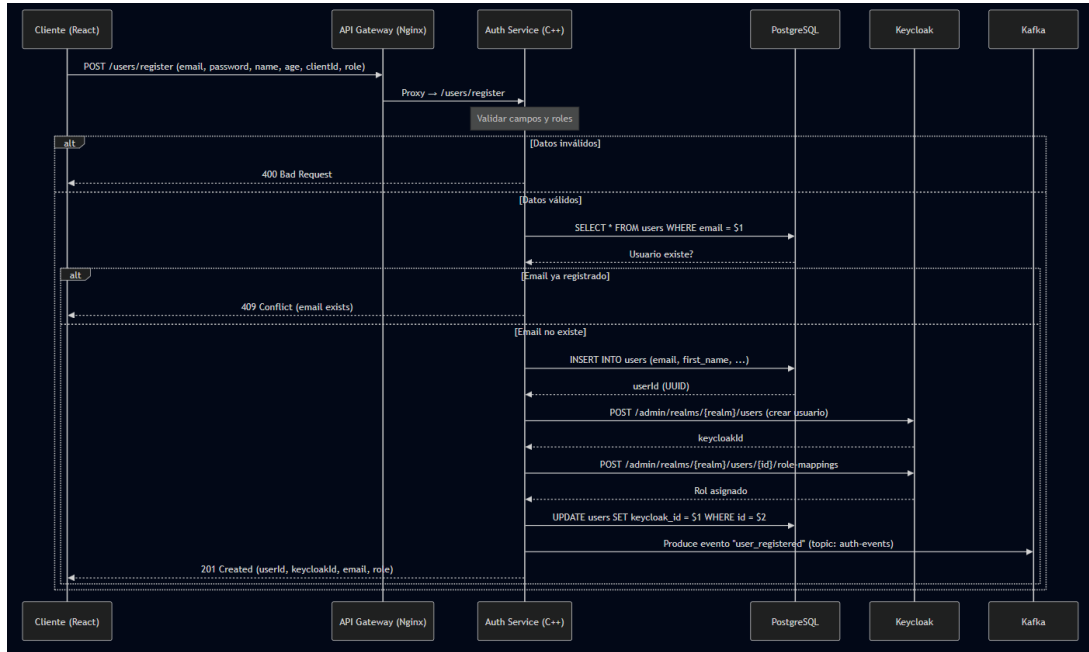


Figure 3: Diagrama de secuencia - Registro de usuario

5.2.3 Proceso 2: Inicio de Sesión

Este diagrama ilustra el proceso de autenticación de un usuario existente. Incluye la verificación de credenciales contra Keycloak, la generación de tokens JWT (access y refresh), el almacenamiento en caché Redis (TTL 300 segundos) y la emisión de eventos a Kafka para auditoría.

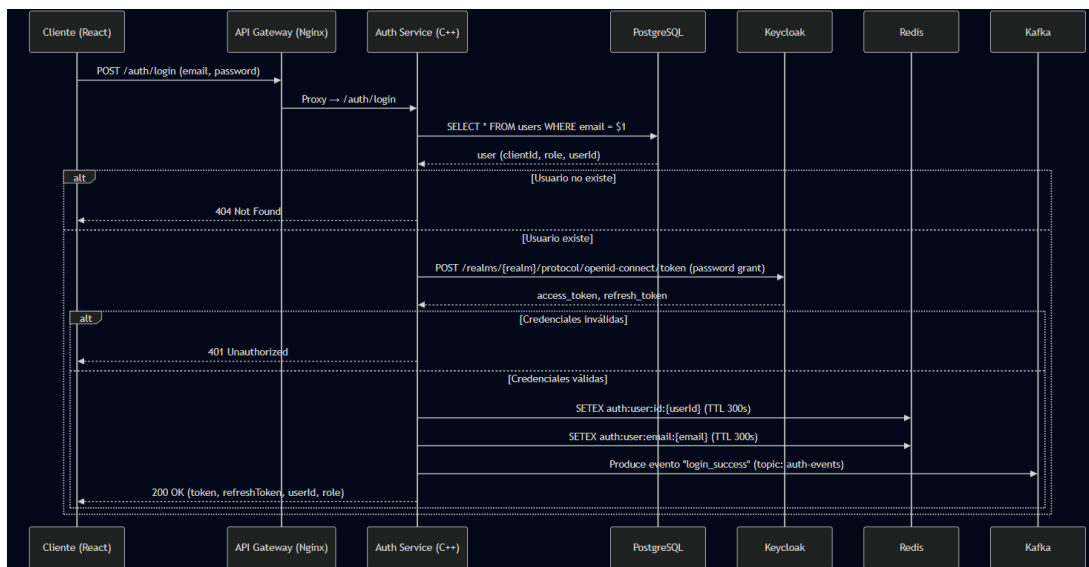


Figure 4: Diagrama de secuencia - Inicio de sesión

5.2.4 Proceso 3: Crear Post con Embedding Semántico

Este diagrama muestra la creación de un post en la comunidad, incluyendo la generación de embeddings semánticos mediante ONNX Runtime. El proceso incluye validación de token, preparación del texto para embedding (título + contenido + etiquetas), inferencia del modelo all-MiniLM-L6-v2 (384 dimensiones), almacenamiento en PostgreSQL con pgvector, subida de imágenes a Cloudinary (opcional) e invalidación de cachés Redis.

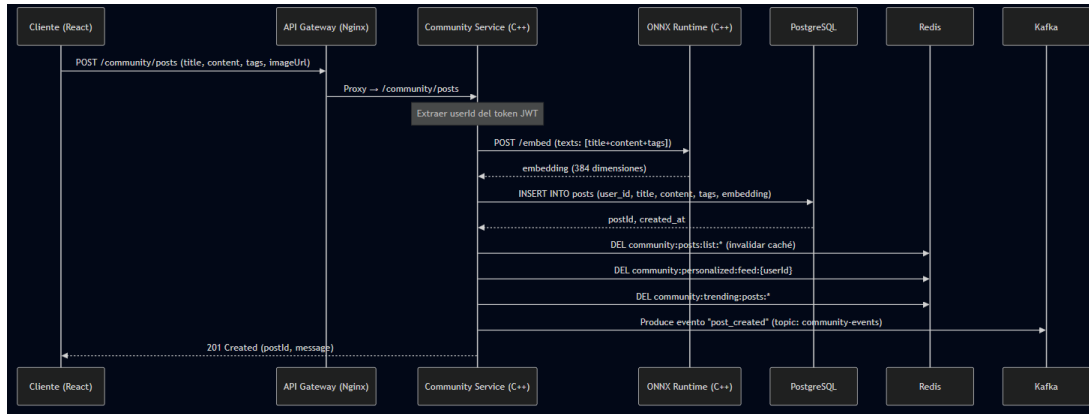


Figure 5: Diagrama de secuencia - Crear post con embedding

5.2.5 Proceso 4: Registrar Pago de Deuda

Este diagrama ilustra el registro de un pago asociado a una deuda. El proceso involucra a los servicios Debt, Transaction y Wallet, actualizando balances y creando transacciones automáticas. Incluye cálculo de intereses, actualización del saldo restante, ajuste del balance de la wallet asociada y emisión de eventos a Kafka.

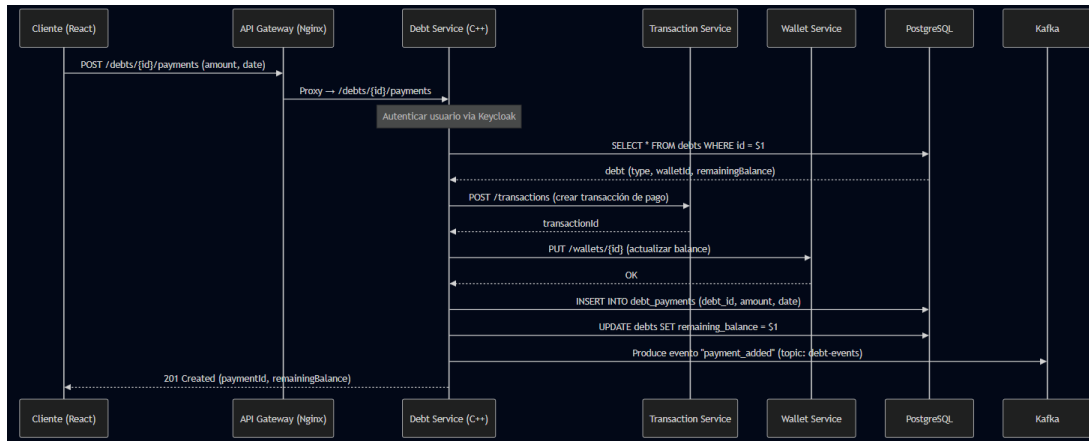


Figure 6: Diagrama de secuencia - Registrar pago de deuda

5.2.6 Proceso 5: Eventos Asíncronos con Kafka

Este diagrama muestra la arquitectura de eventos asíncronos del sistema. Los microservicios Auth y Category producen eventos a los tópicos `auth-events` y `category-events` respectivamente. Estos eventos persisten en Kafka para futuros consumidores como notificaciones push, análisis de datos o sincronización en caliente.

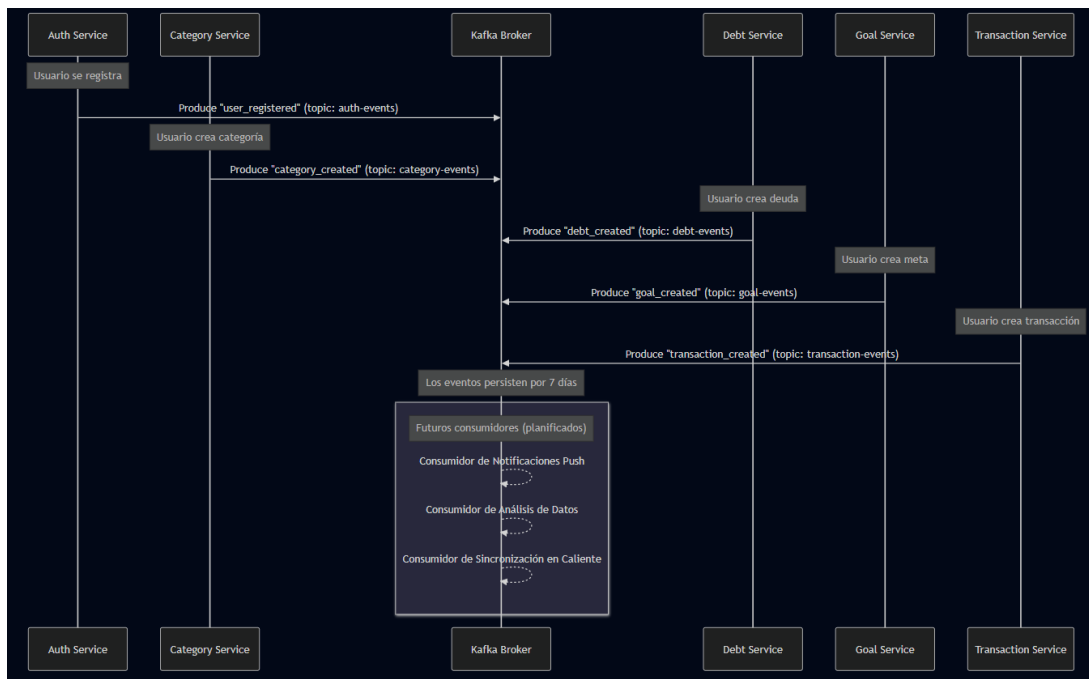


Figure 7: Diagrama de eventos asíncronos - Kafka

5.2.7 Proceso 6: Verificación de Token (Middleware)

Este diagrama muestra el proceso de verificación de tokens JWT que ocurre en cada petición a los microservicios. Incluye extracción del token, verificación local de expiración, caché en Redis (TTL 60 segundos), introspection contra Keycloak en caso de cache miss, y validación del campo `postgresId`. Este proceso es crítico para la seguridad del sistema.

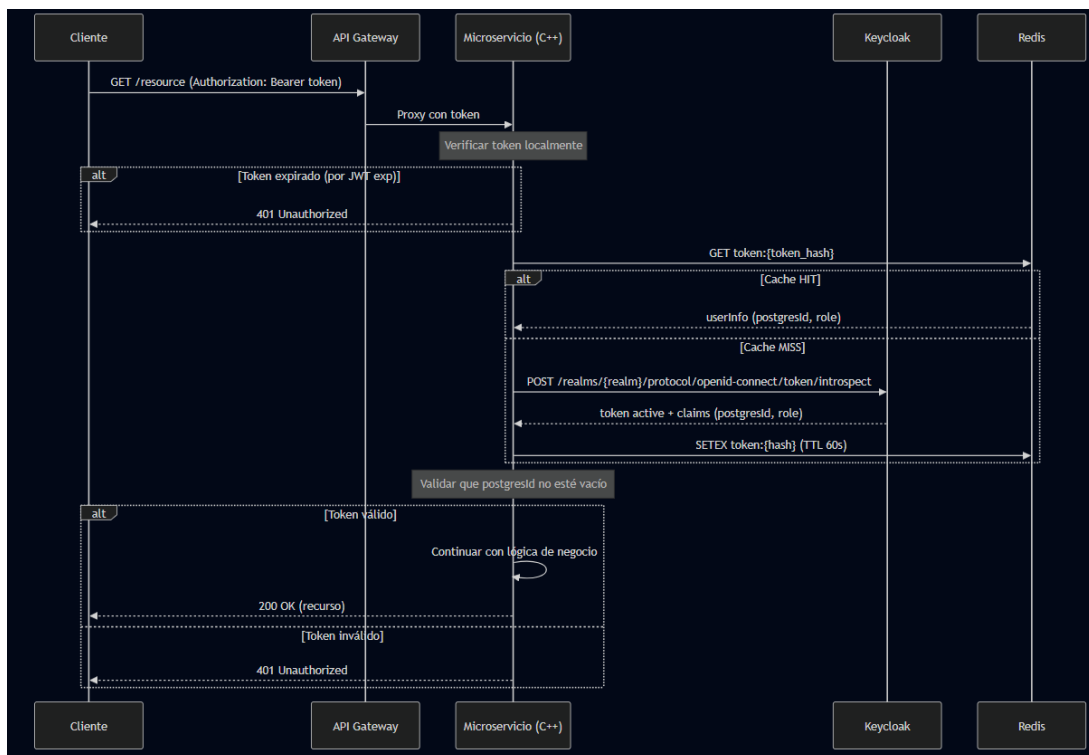


Figure 8: Diagrama de secuencia - Verificación de token

5.3 Vista de Desarrollo

5.3.1 Descripción

La vista de desarrollo representa la organización estática del software en cinco capas tecnológicas, mostrando cómo los componentes se relacionan entre sí desde una perspectiva de desarrollo. Cada capa tiene una responsabilidad bien definida y la regla de diseño establece que un subsistema solo puede depender de subsistemas en la misma capa o en capas inferiores.

Las capas son:

1. **Capa 1 - Frontend (React SPA):** Aplicación React 19 con TypeScript, Zustand para estado, Axios para comunicación HTTP, React Router para navegación y Chart.js para gráficos.
2. **Capa 2 - API Gateway (Nginx):** Proxy inverso que termina SSL (TLS 1.3), enruta peticiones a microservicios y sirve el frontend estático.
3. **Capa 3 - Microservicios (C++ 17):** Nueve microservicios de negocio más Keycloak y ONNX Runtime, desarrollados en C++ con Boost.Beast/Asio.
4. **Capa 4 - Librerías Compartidas (C++):** Componentes reutilizables: conexión a PostgreSQL (libpqxx + pool), cliente Keycloak (libcurl + Boost.JSON), productor Kafka (librdkafka) y cliente Redis (hiredis).
5. **Capa 5 - Infraestructura:** Servicios de soporte: PostgreSQL 16 con pgvector, PgBouncer (pool de conexiones), Redis 7 (caché), Keycloak 22 (IAM), Kafka + Zookeeper (event bus).
6. **Capa 6 - Despliegue:** Docker + Docker Compose sobre Oracle Cloud Infrastructure, con certificados SSL de Let's Encrypt.

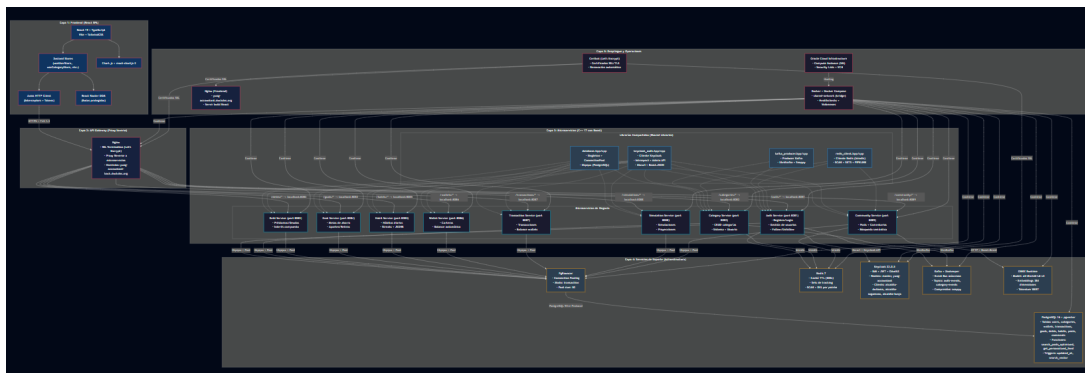


Figure 9: Diagrama de desarrollo - Capas tecnológicas y dependencias

5.4 Vista Física

5.4.1 Descripción

La vista física describe la infraestructura de telecomunicaciones y despliegue del sistema en Oracle Cloud Infrastructure. Muestra los dominios DNS, los certificados SSL/TLS, los puertos de red, los protocolos de comunicación y la topología de red completa.

Los elementos principales son:

- **Dominios DNS (duckdns.org):**
 - `yung-accountant.duckdns.org` → Frontend React
 - `yung-accountant-back.duckdns.org` → API Gateway
 - `yung-accountant-keycloak.duckdns.org` → Keycloak
- **Certificados SSL/TLS (Let's Encrypt):** Emitidos y renovados automáticamente cada 90 días para los tres dominios.

- **Oracle Cloud Infrastructure:**
 - Virtual Cloud Network (VCN) con Internet Gateway
 - Security Lists que permiten puertos 22 (SSH), 80 (HTTP redirige a HTTPS), 443 (HTTPS)
 - Compute Instance (VM) que aloja todos los contenedores Docker
- **Red interna (Docker bridge network):**
 - **shared-network:** red bridge aislada para comunicación entre contenedores
 - Puertos internos: 5432 (PostgreSQL), 6432 (PgBouncer), 6379 (Redis), 9092 (Kafka), 2181 (Zookeeper), 8080-8090 (microservicios)
- **Protocolos de comunicación:**
 - HTTPS/TLS 1.3 entre cliente y gateway (puerto 443)
 - HTTP entre gateway y microservicios (puertos 8081-8089)
 - PostgreSQL Wire entre microservicios y PgBouncer (puerto 6432)
 - RESP (Redis Serialization Protocol) para caché (puerto 6379)
 - Kafka Binary Protocol para eventos (puerto 9092)

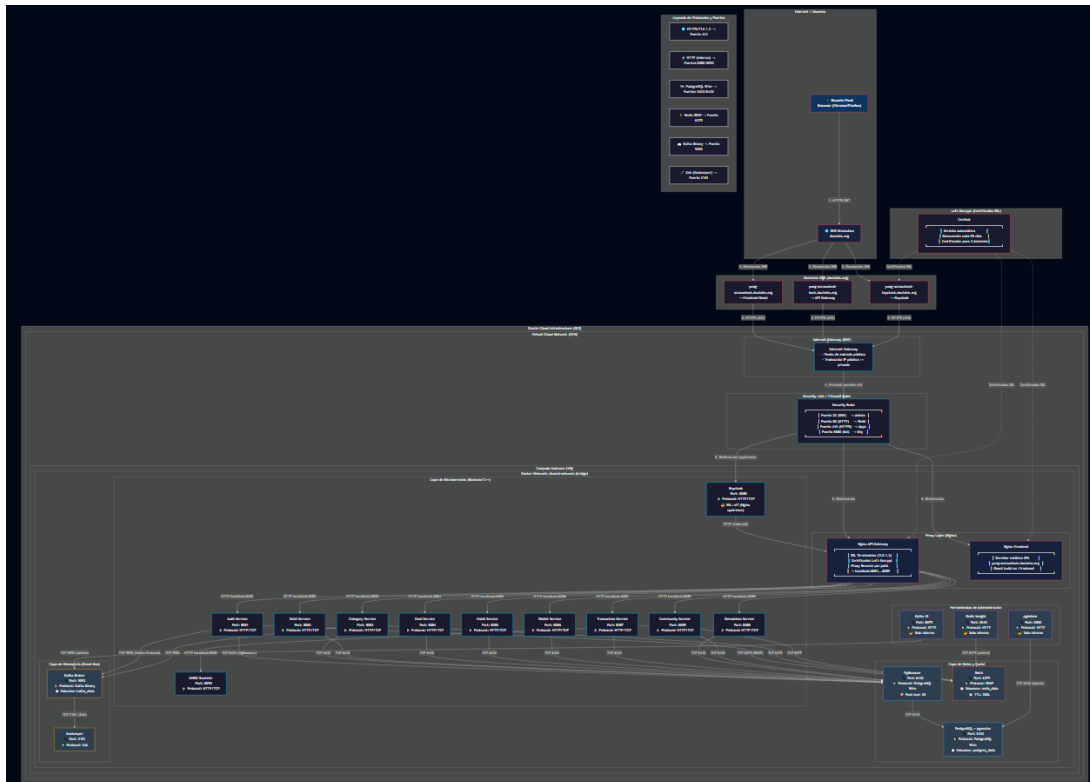


Figure 10: Diagrama físico - Infraestructura de red y despliegue en Oracle Cloud

5.5 Escenarios (+1)

5.5.1 Descripción General

Los escenarios representan casos de uso concretos del sistema, validando la arquitectura mediante ejemplos reales de interacción entre componentes. Cada escenario se documenta con un diagrama de secuencia UML que muestra los actores, los mensajes intercambiados y los componentes involucrados.

5.5.2 Escenario 1: Registro de Usuario

Este escenario describe el proceso completo de registro de un nuevo usuario en el sistema. Incluye validación de campos, verificación de email duplicado, creación en PostgreSQL, registro en Keycloak, asignación de roles y emisión de evento a Kafka. Es fundamental para entender el flujo de onboarding.

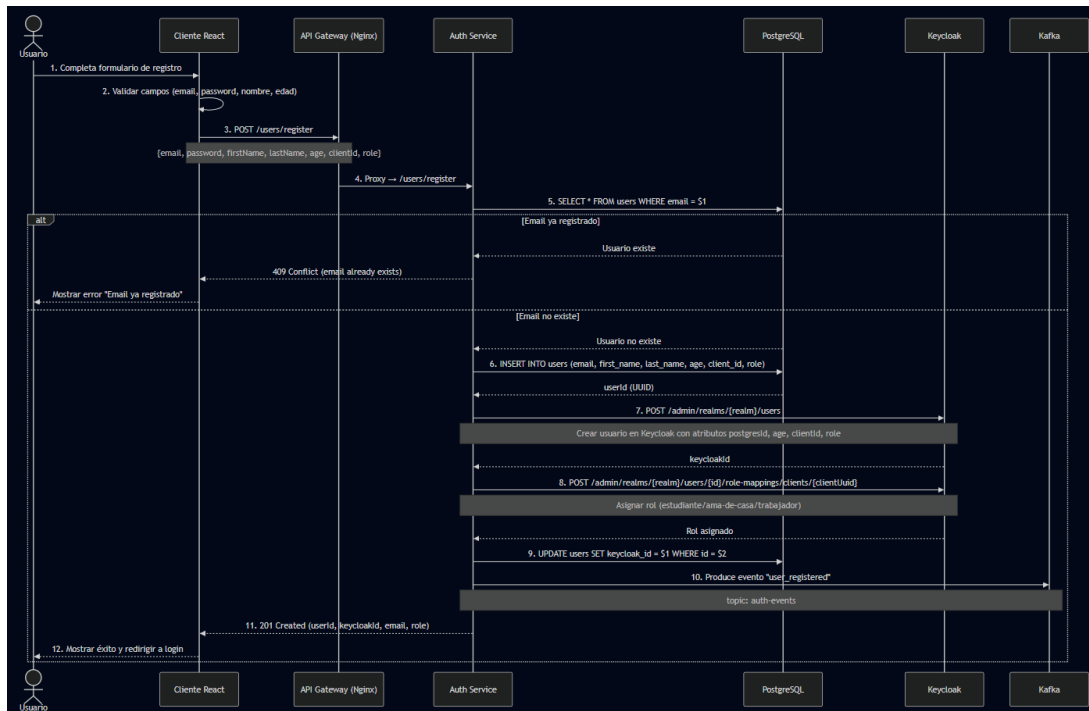


Figure 11: Escenario - Registro de usuario

5.5.3 Escenario 2: Inicio de Sesión

Este escenario muestra la autenticación de un usuario existente mediante email y contraseña. Incluye verificación en Keycloak, generación de tokens JWT, almacenamiento en caché Redis y emisión de evento de auditoría a Kafka.

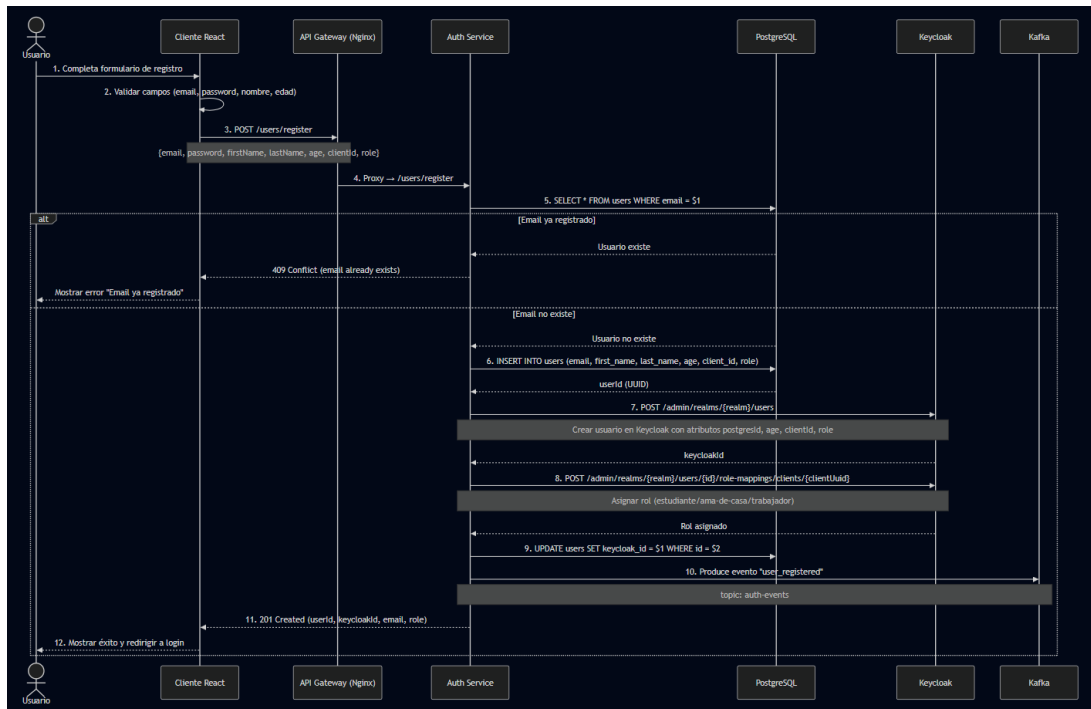


Figure 12: Escenario - Inicio de sesión

5.5.4 Escenario 3: Crear Post con Embedding Semántico

Este escenario ilustra la creación de una publicación en la comunidad. El proceso incluye la generación de un embedding semántico (384 dimensiones) mediante ONNX Runtime, almacenamiento en PostgreSQL con pgvector, subida opcional de imágenes a Cloudinary e invalidación de cachés Redis.

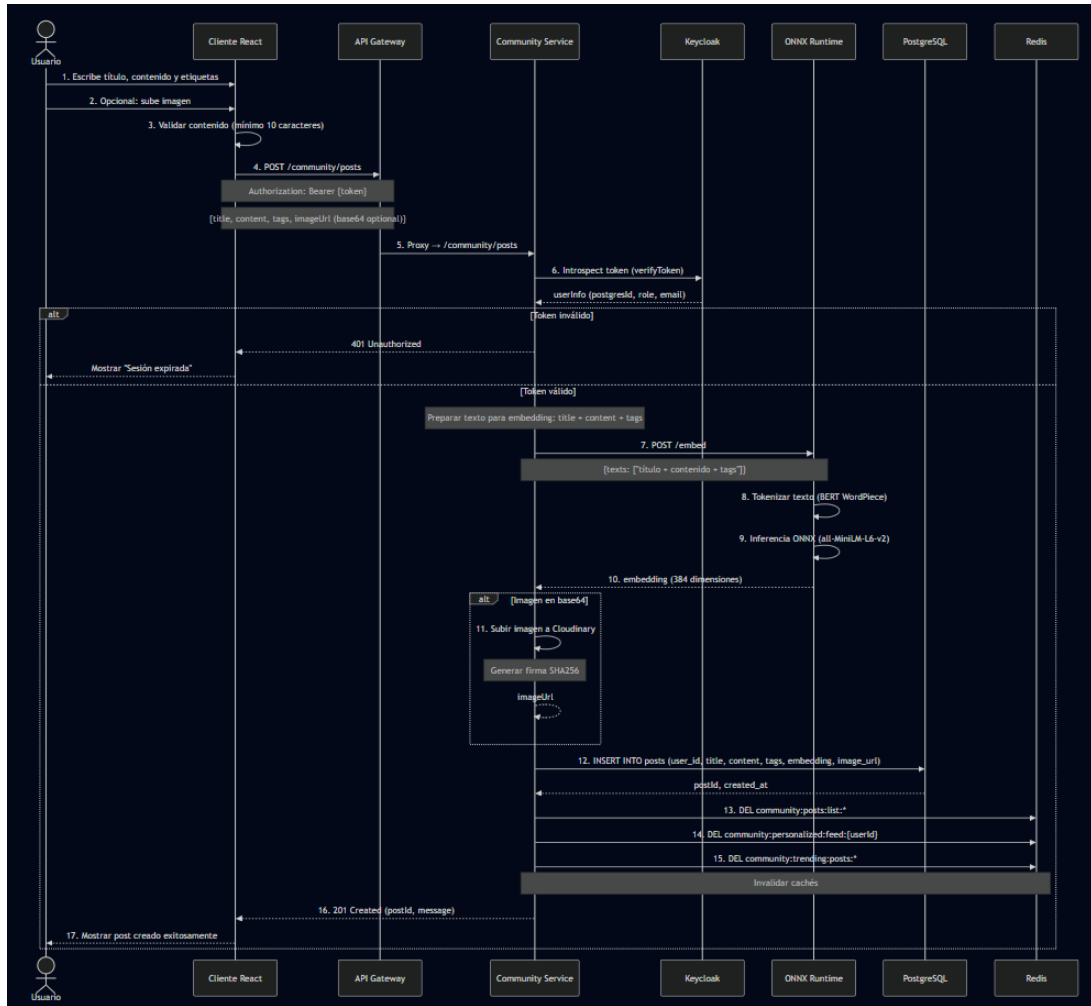


Figure 13: Escenario - Crear post con embedding semántico

5.5.5 Escenario 4: Registrar Pago de Deuda

Este escenario muestra el registro de un pago asociado a una deuda. Involucra a los servicios Debt, Transaction y Wallet, actualizando balances y creando transacciones automáticas. Es un ejemplo de transacción distribuida entre microservicios.

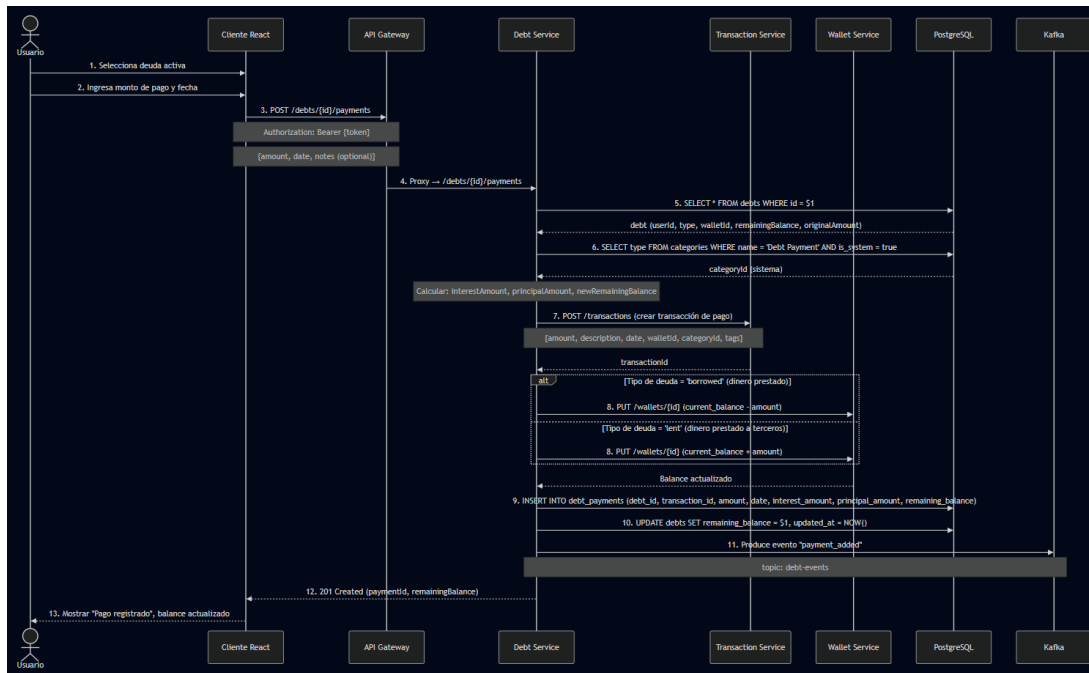


Figure 14: Escenario - Registrar pago de deuda

5.5.6 Escenario 5: Feed Personalizado

Este escenario describe la obtención del feed personalizado del usuario. Utiliza una función PL/pgSQL optimizada (`get_personalized_feed`) que calcula un score de relevancia basado en seguidos (prioridad +10), likes (+1.5), comentarios (+2.0), tags coincidentes (+3.0) y recencia (factor 0.3). Los resultados se cachean en Redis con TTL de 120 segundos.

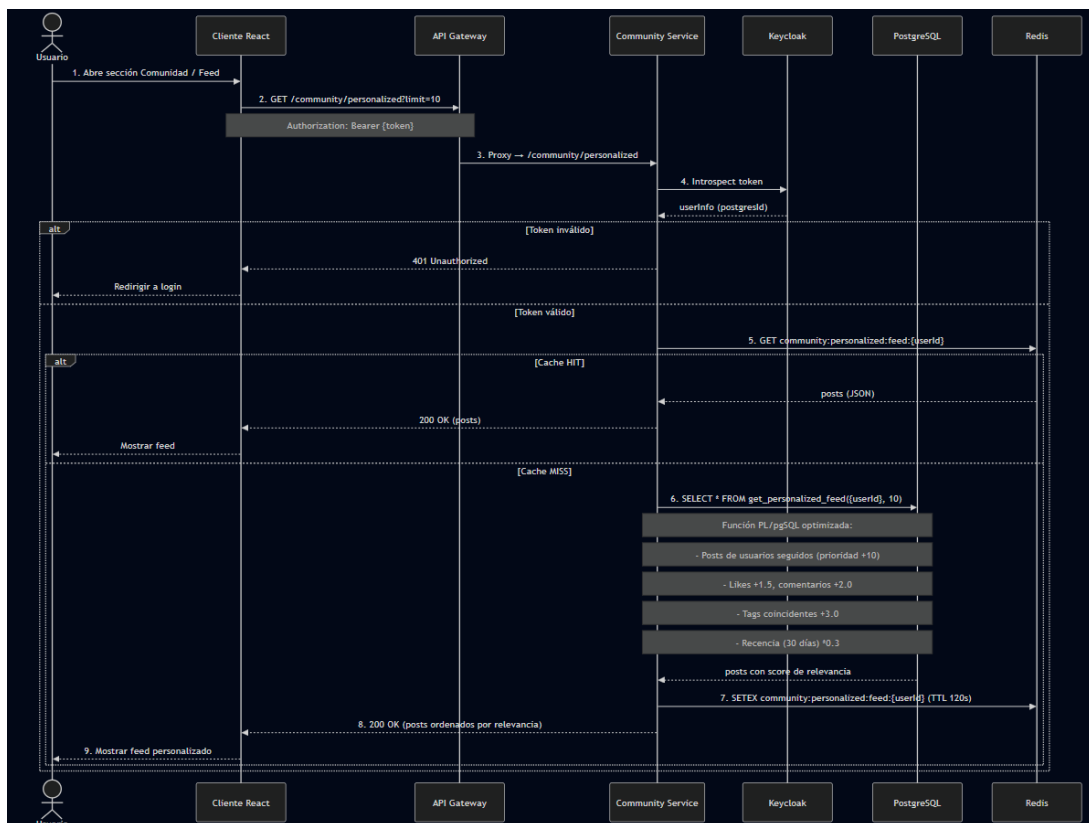


Figure 15: Escenario - Feed personalizado

5.5.7 Escenario 6: Búsqueda de Usuarios

Este escenario ilustra la búsqueda de usuarios mediante la función `search_users_trgm`, que utiliza el operador de similitud trigrama (%) de PostgreSQL. La búsqueda compara `username`, `display_name`, `first_name` y `last_name`, ordenando los resultados por similitud descendente. Los resultados se cachean en Redis con TTL de 120 segundos.

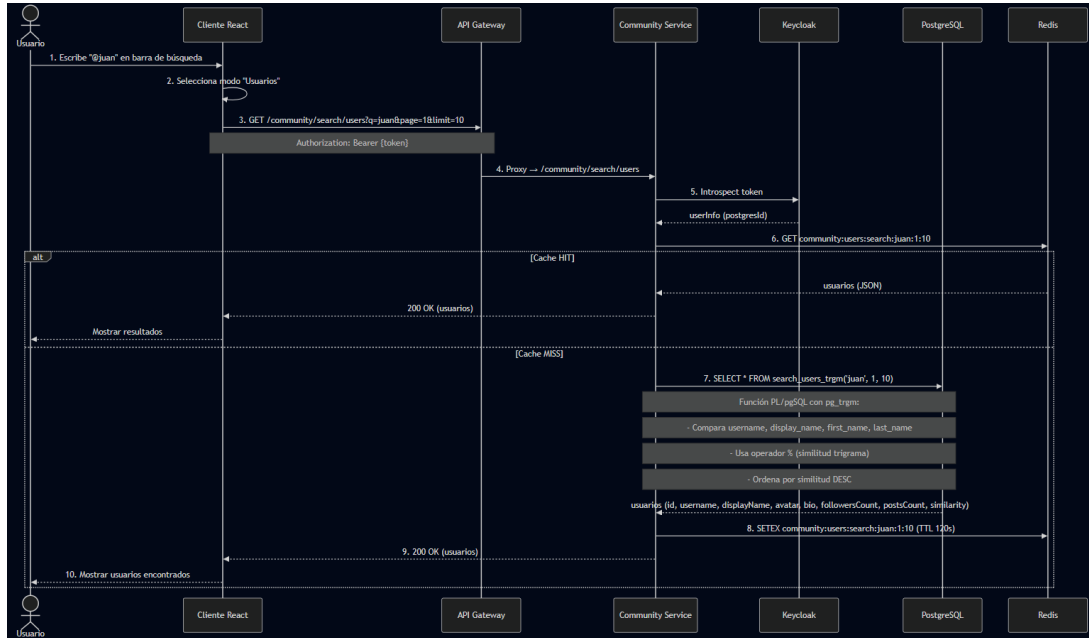


Figure 16: Escenario - Búsqueda de usuarios

6 Modelo Entidad-Relación (MER)

El modelo entidad-relación describe la estructura lógica de la base de datos del sistema Yung Accountant. Define las entidades principales, sus atributos y las relaciones entre ellas, proporcionando una visión clara del esquema de datos utilizado por todos los microservicios.

6.1 Descripción General

La base de datos del sistema está implementada en PostgreSQL 16 con la extensión `pgvector` para soporte de búsqueda semántica. El modelo consta de **15 tablas principales** que almacenan toda la información del sistema: usuarios, categorías, transacciones, carteras, metas, deudas, hábitos, publicaciones, comentarios e interacciones.

6.2 Entidades Principales

6.2.1 User (Usuarios)

Tabla central que almacena la información de perfil de cada usuario. Se relaciona con Keycloak mediante el campo `keycloak_id`, que vincula la identidad en PostgreSQL con el Identity Provider. Los campos `followers` y `following` son arreglos de UUID que permiten el seguimiento entre usuarios sin necesidad de una tabla intermedia.

6.2.2 Category (Categorías)

Gestiona las categorías de ingresos y gastos. Las categorías del sistema tienen `user_id = NULL` y son inmutables, mientras que las categorías de usuario son personalizables. El campo `type` discrimina entre `income` (ingreso) y `expense` (gasto).

6.2.3 Wallet (Carteras)

Representa cuentas financieras del usuario: efectivo, cuentas bancarias, tarjetas de crédito/débito u otros. El campo `current_balance` se actualiza automáticamente mediante triggers desde las transacciones asociadas.

6.2.4 Transaction (Transacciones)

Registra cada movimiento financiero. Está asociada a una wallet (origen/destino del dinero) y a una categoría (que determina si es ingreso o gasto). El campo `tags` es un arreglo de strings para etiquetado flexible.

6.2.5 Goal (Metas)

Define objetivos de ahorro con monto objetivo, fecha límite, prioridad y estado. El campo `current_amount` se actualiza mediante transacciones de meta (`goal_transactions`).

6.2.6 Debt (Deudas)

Registra préstamos recibidos (`type = 'borrowed'`) o prestados a terceros (`type = 'lent'`). Soporta interés fijo o variable, y capitalización compuesta mediante el campo `compound_months`.

6.2.7 Habit (Hábitos)

Almacena hábitos diarios con seguimiento de rachas (`current_streak`, `best_streak`). Los checks diarios se almacenan como JSONB en el campo `checks`, permitiendo flexibilidad en el formato.

6.2.8 Post (Publicaciones)

Contenido generado por usuarios en la comunidad. Incluye campos para búsqueda semántica: `embedding vector(384)` para almacenar el embedding generado por ONNX Runtime, y `search_vector tsvector` para búsqueda full-text.

6.2.9 Comment (Comentarios)

Comentarios y respuestas anidadas a publicaciones. El campo `parent_id` permite crear jerarquías de respuestas (auto-referencia).

6.3 Relaciones Clave

Las principales relaciones entre entidades son:

- **User → Category:** Uno a muchos. Un usuario puede tener múltiples categorías personalizadas.
- **User → Wallet:** Uno a muchos. Un usuario puede tener múltiples carteras.
- **User → Transaction:** Uno a muchos. Un usuario realiza muchas transacciones.
- **User → Goal:** Uno a muchos. Un usuario establece múltiples metas.
- **User → Debt:** Uno a muchos. Un usuario puede tener múltiples deudas.
- **User → Habit:** Uno a muchos. Un usuario tiene múltiples hábitos.
- **User → Post:** Uno a muchos. Un usuario escribe múltiples posts.
- **User → Comment:** Uno a muchos. Un usuario escribe múltiples comentarios.
- **Category → Transaction:** Uno a muchos. Una categoría clasifica muchas transacciones.
- **Wallet → Transaction:** Uno a muchos. Una wallet registra muchas transacciones.
- **Goal → GoalTransaction:** Uno a muchos (composición). Una meta tiene muchas transacciones asociadas.

- **Transaction** → **GoalTransaction**: Uno a muchos (dependencia). Una transacción puede estar vinculada a una transacción de meta.
- **Debt** → **DebtPayment**: Uno a muchos (composición). Una deuda tiene muchos pagos.
- **Debt** → **VariableInterest**: Uno a muchos (composición). Una deuda con interés variable tiene tasas por mes.
- **Transaction** → **DebtPayment**: Uno a muchos (dependencia). Una transacción puede ser un pago de deuda.
- **Post** → **Comment**: Uno a muchos (composición). Un post tiene muchos comentarios.
- **Comment** → **Comment**: Auto-referencia (0..1 a muchos). Un comentario puede tener muchas respuestas.
- **Post** → **UserPostInteraction**: Uno a muchos. Un post tiene muchas interacciones de usuarios.
- **User** → **UserPostInteraction**: Uno a muchos. Un usuario tiene muchas interacciones con posts.

7 Microservicios de Infraestructura

7.1 Servicio de Base de Datos (PostgreSQL + pgvector)

7.1.1 Descripción General

El sistema utiliza PostgreSQL 16 con la extensión **pgvector** como motor de base de datos principal. Proporciona almacenamiento persistente, consistencia transaccional y soporte para búsquedas vectoriales (embeddings). Se encarga de gestionar todas las tablas del sistema: usuarios, categorías, transacciones, wallets, metas, deudas, hábitos, posts, comentarios e interacciones.

7.1.2 ¿Qué hace?

- Almacena toda la información estructurada del sistema Yung Accountant.
- Mantiene relaciones entre entidades mediante claves foráneas (ej. `transactions.user_id` → `users.id`).
- Proporciona funciones PL/pgSQL para búsqueda híbrida (full-text + semántica) mediante **pgvector**.
- Gestiona triggers para actualización automática de timestamps (`updated_at`) y vectores de búsqueda (`search_vector`).
- Asegura integridad referencial con eliminaciones en cascada (`ON DELETE CASCADE`) donde corresponde.

7.1.3 ¿Cómo lo hace?

- Utiliza `pgvector/pgvector:pg16` como imagen Docker, que incluye la extensión vectorial.
- Las conexiones se realizan a través de **PgBouncer** (pool de conexiones en modo `transaction`), que actúa como proxy entre los microservicios y PostgreSQL.
- Cada microservicio utiliza la librería `libpqxx` (C++) para ejecutar consultas parametrizadas, previniendo inyección SQL.
- La tabla `posts` incluye una columna `embedding vector(384)` para almacenar los vectores generados por el servicio ONNX, y `search_vector tsvector` para búsqueda full-text.
- Las funciones almacenadas (`search_posts_optimized`, `get_personalized_feed`, `search_users_trgm`) implementan algoritmos de búsqueda avanzados.

7.1.4 Integración

- **Microservicios:** Todos los servicios (`auth`, `category`, `debt`, `goal`, `habit`, `wallet`, `transaction`, `simulation`, `community`) se conectan mediante `Database::getInstance().connect()`.
- **PgBouncer:** Actúa como intermediario en el puerto 6432, gestionando un pool de hasta 50 conexiones a PostgreSQL.
- **Redis:** La caché de segundo nivel complementa a PostgreSQL, reduciendo la carga de consultas repetitivas.
- **pgAdmin:** Interfaz gráfica disponible en el puerto 5050 para administración y monitoreo.

7.1.5 Justificación

- **pgvector:** Necesaria para búsqueda semántica de posts (embeddings de 384 dimensiones), permitiendo recomendaciones personalizadas y búsqueda por similitud.
- **PgBouncer:** Optimiza el uso de conexiones, evitando el overhead de crear/destruir conexiones por cada petición HTTP.
- **Triggers y funciones PL/pgSQL:** Descargan lógica de búsqueda y actualización al motor de base de datos.

7.1.6 Alcance

- Gestiona **15 tablas principales**.
- Almacena aproximadamente **20 categorías del sistema** predefinidas.
- Soporta **búsquedas híbridas:** combinación de coincidencia de palabras clave (`pg_trgm`) y similitud semántica (`pgvector`).
- Proporciona **función de feed personalizado** que prioriza posts de usuarios seguidos.

7.1.7 Configuración de Entorno

- `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`
- `POSTGRES_HOST` (generalmente `pgbouncer`)
- `POSTGRES_PORT` (5432 para directo, 6432 para PgBouncer)
- Volúmenes persistentes: `postgres_data`, `pgadmin_data`

7.2 Servicio de Caché (Redis)

7.2.1 Descripción General

Redis es el servicio de caché distribuida en memoria del sistema Yung Accountant. Actúa como un almacenamiento de clave-valor de alto rendimiento que reduce la carga en PostgreSQL y mejora la latencia de respuestas.

7.2.2 ¿Qué hace?

- Almacena en caché objetos frecuentemente consultados: perfiles de usuario, listas de categorías.
- Reduce consultas a PostgreSQL, disminuyendo latencia y consumo de conexiones.
- Proporciona operaciones atómicas sobre conjuntos (sets) para rastrear claves en caché.
- Permite invalidación selectiva mediante patrones (`SCAN + DELETE`).
- Soporta operaciones de incremento atómico (`INCRBY`) para contadores.

7.2.3 ¿Cómo lo hace?

- Corre sobre la imagen `redis:7-alpine` con persistencia AOF habilitada.
- Requiere autenticación mediante contraseña configurada en `REDIS_PASSWORD`.
- Cada microservicio implementa un cliente Redis singleton con manejo thread-safe.
- Las claves siguen patrones predefinidos para facilitar invalidación.
- Se mantienen **sets de seguimiento** que contienen todas las claves activas de un tipo.

7.2.4 Integración

- **Servicio auth:** Cachea objetos `User` completos con TTL de 300 segundos.
- **Servicio category:** Cachea listas de categorías del sistema y de usuario.
- **Redis Insight:** Interfaz gráfica en puerto 5540 para monitoreo.

7.2.5 Justificación

- **Rendimiento:** Redis opera en memoria con latencia submilisegundo.
- **Escalabilidad:** Reduce la carga en PostgreSQL para operaciones de lectura repetitivas.
- **Invalidación inteligente:** Los sets de seguimiento permiten invalidación masiva eficiente.
- **TTL automático:** Las claves expiran después de 5 minutos.

7.2.6 Alcance

- Cachea **objetos de usuario** (1 KB cada uno).
- Cachea **categorías del sistema** (20 registros) y **categorías de usuario**.
- Soporta operaciones de **conjuntos** (`SADD`, `SREM`, `SMEMBERS`).
- Implementa `delSet()` para eliminación atómica de sets completos.

7.2.7 Configuración de Entorno

- `REDIS_HOST` (por defecto `redis`)
- `REDIS_PORT` (por defecto `6379`)
- `REDIS_PASSWORD` (por defecto `redis123`)
- Volúmenes persistentes: `redis_data`, `redis_insight_data`

7.3 Servicio de Autenticación y Autorización (Keycloak)

7.3.1 Descripción General

Keycloak es el servicio de gestión de identidades y acceso (IAM) del sistema. Actúa como servidor de autenticación centralizado, emisor de tokens JWT, gestor de usuarios y roles, y proveedor de single sign-on (SSO).

7.3.2 ¿Qué hace?

- Autentica usuarios mediante email/contraseña, retornando `access_token` y `refresh_token`.
- Verifica la validez de tokens JWT mediante el endpoint de introspection.
- Gestiona el ciclo de vida completo de usuarios: creación, actualización, eliminación.
- Almacena atributos personalizados (`postgresId`, `age`, `clientId`, `role`) en el token JWT.
- Maneja cierre de sesión y revocación de tokens.

7.3.3 ¿Cómo lo hace?

- Corre sobre `quay.io/keycloak/keycloak:22.0.5` en modo desarrollo.
- Expone el puerto 8080 internamente, con Nginx como proxy HTTPS.
- Utiliza `admin-cli` como cliente administrativo.
- Define tres clientes (alcaldías) con sus respectivos secrets.
- Los roles disponibles: `estudiante`, `ama-de-casa`, `trabajador`.

7.3.4 Integración

- **Servicio auth:** Cliente completo con operaciones administrativas.
- **Demás microservicios:** Versión reducida solo para verificación de tokens.
- **Frontend:** Recibe y utiliza el token JWT en todas las peticiones.
- **Nginx:** Proxy inverso con terminación SSL.

7.3.5 Justificación

- **Seguridad:** Las contraseñas nunca tocan los microservicios.
- **Estandarización:** Implementa OpenID Connect y OAuth2.
- **Atributos embebidos:** `postgresId` en el token evita consultas adicionales.
- **Escalabilidad:** Verificación por introspection sin compartir secretos.

7.3.6 Alcance

- Gestiona **2 realms**: `master` y `yung-accountant`.
- Define **3 clientes** (Alcaldía de Sogamoso, Duitama, Tunja) con sus respectivos **roles** (Estudiante, Ama de Casa, Trabajador).
- Soporta **tokens JWT** con expiración configurable.

7.3.7 Configuración de Entorno

- `KEYCLOAK_ADMIN`, `KEYCLOAK_ADMIN_PASSWORD`
- `KC_HOSTNAME` (dominio público)
- `KC_HTTP_ENABLED=true`

7.4 Servicio de Mensajería y Eventos (Kafka)

7.4.1 Descripción General

Apache Kafka es el bus de eventos asíncrono del sistema. Permite la comunicación desacoplada entre microservicios mediante el patrón productor-consumidor.

7.4.2 ¿Qué hace?

- Recibe eventos de los microservicios y los almacena en tópicos.
- Permite que múltiples consumidores lean eventos de un mismo tópico.
- Proporciona tolerancia a fallos: los eventos persisten en disco.

7.4.3 ¿Cómo lo hace?

- Utiliza **Zookeeper** para gestión de clúster.
- El **broker Kafka** (Confluent 7.5.0) expone el puerto 9092.
- Los productores usan **librdkafka** para enviar mensajes JSON.
- Actualmente solo hay productores; los consumidores están planificados.

7.4.4 Integración

- **Servicio auth**: Produce eventos al tópico `auth-events`.
- **Servicio category**: Produce eventos al tópico `category-events`.
- **Kafka UI**: Interfaz gráfica en puerto 8079.

7.4.5 Justificación

- **Desacoplamiento**: Los microservicios no necesitan conocerse entre sí.
- **Trazabilidad**: Los eventos proporcionan una pista de auditoría completa.
- **Escalabilidad futura**: Permite agregar consumidores para notificaciones.

7.4.6 Alcance

- Define **9 tópicos activos**: `auth-events`, `category-events`, `community-events`, `debt-events`, `goal-events`, `habit-events`, `simulation-events`, `transaction-events` y `wallet-events`.
- Soporta **compresión snappy**.
- Proporciona **Kafka UI** para depuración.

7.4.7 Configuración de Entorno

- `KAFKA_BROKER` (ej. `kafka:9092`)
- Volúmenes: `zookeeper_data`, `kafka_data`

7.5 Servicio de Embeddings (ONNX Runtime)

7.5.1 Descripción General

El servicio `embedding-onnx` genera vectores numéricos (embeddings) de 384 dimensiones a partir de texto, utilizando el modelo `all-MiniLM-L6-v2` y ONNX Runtime para inferencia eficiente en C++.

7.5.2 ¿Qué hace?

- Convierte texto en vectores de 384 dimensiones.
- Normaliza los embeddings con pooling mean y normalización L2.
- Expone un servidor HTTP para recibir lotes de textos.

7.5.3 ¿Cómo lo hace?

- El modelo se descarga desde HuggingFace durante la construcción Docker.
- Se exporta a ONNX con opset 15 y batch dinámico.
- El servicio C++ carga el modelo con ONNX Runtime e inicializa el tokenizer.
- Expone `POST /embed` y `GET /health`.

7.5.4 Integración

- **Servicio community:** Se conecta a `http://embedding-onnx:8090`.
- Los embeddings se almacenan en `posts.embedding` (pgvector).
- Búsquedas semánticas con función `search_posts_semantic`.

7.5.5 Justificación

- **Rendimiento:** ONNX Runtime en C++ es mucho más rápido que Python.
- **Aislamiento:** Microservicio independiente y escalable.
- **Modelo ligero:** 22.7 MB, embeddings de 384 dimensiones.
- **Búsqueda semántica:** Encuentra posts por significado, no solo palabras clave.

7.5.6 Alcance

- Soporta **batch processing** de textos.
- Tokenizador con vocabulario de 30,000 tokens y longitud máxima de 128.
- Servicio con recursos limitados (512 MB de RAM).

7.5.7 Configuración de Entorno

- `MODEL_PATH`, `TOKENIZER_PATH`
- `NUM_THREADS` (por defecto 4)
- `PORT` (por defecto 8090)
- Volumen: `embedding_models`

8 Microservicios de Negocio

8.1 Servicio de Autenticación (auth)

8.1.1 Descripción General

El servicio `auth` gestiona el registro, autenticación y autorización de usuarios. Actúa como punto de entrada para registro, login, logout, perfiles, seguidores y operaciones administrativas. Se expone a través del API Gateway en `https://yung-accountant-back.duckdns.org`.

8.1.2 Endpoints de la API REST

Registro de usuario Registra un nuevo usuario en PostgreSQL y Keycloak.

Path	<code>/users/register</code>
Método	POST
Parámetros	<code>email</code> , <code>password</code> , <code>firstName</code> , <code>lastName</code> , <code>age</code> , <code>clientId</code> , <code>role</code>
Retorno	<code>userId</code> , <code>keycloakId</code> , <code>email</code> , <code>firstName</code> , <code>lastName</code> , <code>clientId</code> , <code>role</code>

Inicio de sesión Autentica usuario y retorna tokens JWT.

Path	<code>/auth/login</code>
Método	POST
Parámetros	<code>email</code> , <code>password</code>
Retorno	<code>token</code> , <code>refreshToken</code> , <code>userId</code> , <code>email</code> , <code>firstName</code> , <code>lastName</code> , <code>clientId</code> , <code>role</code>

Cierre de sesión Invalida refresh token y cierra sesiones activas.

Path	/auth/logout
Método	POST
Parámetros	refreshToken
Retorno	message, refreshInvalidated, sessionsClosed

Obtener perfil propio Retorna información completa del usuario autenticado.

Path	/users/me
Método	GET
Parámetros	Token en cabecera Authorization
Retorno	Objeto completo del usuario autenticado

Actualizar perfil Actualiza datos del perfil del usuario autenticado.

Path	/users/update
Método	PUT
Parámetros	firstName, lastName, age, bio, location, website, profilePic, clientId, role
Retorno	message, keycloakUpdated

Eliminar cuenta Elimina usuario de PostgreSQL y Keycloak.

Path	/users/delete
Método	DELETE
Parámetros	Token en cabecera Authorization
Retorno	message

Seguir a un usuario Agrega un usuario a la lista de seguidos.

Path	/users/follow
Método	POST
Parámetros	userId
Retorno	message

Dejar de seguir a un usuario Remueve un usuario de la lista de seguidos.

Path	/users/unfollow
Método	POST
Parámetros	userId
Retorno	message

Obtener usuario por email Retorna información pública por email.

Path	/users/by-email/{email}
Método	GET
Parámetros	email en URL
Retorno	Objeto público del usuario

Obtener usuario por username Retorna información pública por nombre de usuario.

Path	/users/by-username/{username}
Método	GET
Parámetros	username en URL
Retorno	Objeto público del usuario

Listar todos los usuarios Retorna lista completa de usuarios.

Path	/users
Método	GET
Parámetros	-
Retorno	Lista de todos los usuarios

Listar clientes disponibles Retorna los clientes (entidades) disponibles.

Path	/clients
Método	GET
Parámetros	-
Retorno	alcaldia-duitama, alcaldia-sogamoso, alcaldia-tunja

Listar roles disponibles Retorna los roles disponibles.

Path	/roles
Método	GET
Parámetros	-
Retorno	estudiante, ama-de-casa, trabajador

Invalidar caché Invalida caché de Redis para el usuario.

Path	/cache/invalidate
Método	POST
Parámetros	<i>Token en cabecera Authorization</i>
Retorno	message

Verificar salud del servicio Health check del servicio.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Los endpoints requieren autenticación con Authorization: Bearer <token>.

8.2 Servicio de Categorías (category)

8.2.1 Descripción General

El servicio `category` gestiona las categorías de ingresos y gastos. Permite obtener categorías del sistema (inmutables) y categorías personalizadas por usuario. Se comunica con PostgreSQL, Redis, Kafka y Keycloak.

8.2.2 Endpoints de la API REST

Obtener categorías del sistema Retorna categorías predefinidas (compartidas globalmente).

Path	/categories/system
Método	GET
Parámetros	<i>Ninguno</i>
Retorno	Lista de categorías del sistema

Obtener categorías del usuario Retorna categorías personalizadas del usuario autenticado.

Path	/categories/user
Método	GET
Parámetros	<i>Token en cabecera</i>
Retorno	Lista de categorías del usuario

Obtener todas las categorías Combina categorías del sistema y del usuario.

Path	/categories/all
Método	GET
Parámetros	<i>Token en cabecera</i>
Retorno	Lista combinada de todas las categorías

Obtener categoría por ID Retorna categoría específica por ID.

Path	/categories/{id}
Método	GET
Parámetros	<i>id en URL, Token en cabecera</i>
Retorno	Objeto de la categoría

Crear categoría de usuario Crea nueva categoría personalizada.

Path	/categories/user
Método	POST
Parámetros	<i>name, type, icon, color, isDefault</i>
Retorno	<i>Categoría creada con id, message</i>

Actualizar categoría de usuario Actualiza campos de categoría existente.

Path	/categories/user/{id}
Método	PUT
Parámetros	<i>id en URL, Token, campos a actualizar</i>
Retorno	<i>message, id</i>

Eliminar categoría de usuario Elimina categoría si no tiene transacciones asociadas.

Path	/categories/user/{id}
Método	DELETE
Parámetros	<i>id en URL, Token en cabecera</i>
Retorno	<i>message</i>

Invaldar caché de categorías Invalida caché de Redis para categorías del usuario.

Path	/categories/cache/invalidate
Método	POST
Parámetros	<i>Token en cabecera</i>
Retorno	message

Verificar salud del servicio Health check del servicio.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Los endpoints que modifican datos requieren autenticación con **Authorization: Bearer <token>**.*

8.3 Servicio de Comunidad (community)

8.3.1 Descripción General

El servicio `community` gestiona todo el contenido social del sistema: posts, comentarios, respuestas anidadas, likes, seguimiento de usuarios, búsqueda semántica, feed personalizado y tendencias. Se integra con el servicio `embedding-onnx` para generar embeddings de posts (búsqueda por similitud), con PostgreSQL (tablas `posts`, `comments`, `user.post.interactions`), Redis para caché de feeds y resultados de búsqueda, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.3.2 Endpoints de la API REST

Obtener posts Retorna lista paginada de posts. Puede filtrar solo posts de usuarios seguidos.

Path	/community/posts
Método	GET
Parámetros	page, limit, following (query params, opcionales), <i>Token en cabecera</i>
Retorno	Lista de posts con id, title, content, likesCount, commentsCount, username, displayName, avatar, createdAt, imageUrl, tags

Obtener post por ID Retorna un post específico con todos sus detalles.

Path	/community/posts/{id}
Método	GET
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	Objeto completo del post con likedBy (IDs de usuarios que dieron like)

Crear post Crea un nuevo post. Si contiene imagen en base64, la sube a Cloudinary. Genera embedding automáticamente.

Path	/community/posts
Método	POST
Parámetros	title, content, tags (array), imageUrl (base64 opcional)
Retorno	id, message

Actualizar post Actualiza título, contenido, etiquetas o imagen de un post existente.

Path	/community/posts/{id}
Método	PUT
Parámetros	title, content, tags, imageUrl (todos opcionales), <i>Token</i>
Retorno	message

Eliminar post Elimina un post y su imagen asociada de Cloudinary.

Path	/community/posts/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Dar/Quitar like a post Toggle like en un post.

Path	/community/posts/{id}/like
Método	POST
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Obtener comentarios de un post Retorna todos los comentarios raíz de un post con sus respuestas anidadas.

Path	/community/posts/{id}/comments
Método	GET
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	Lista jerárquica de comentarios con replies anidados

Agregar comentario Añade un comentario a un post.

Path	/community/posts/{id}/comments
Método	POST
Parámetros	content, <i>Token en cabecera</i>
Retorno	id, message

Actualizar comentario Actualiza el contenido de un comentario.

Path	/comments/{id}
Método	PUT
Parámetros	id en URL, content, <i>Token</i>
Retorno	message

Eliminar comentario Elimina un comentario y todas sus respuestas.

Path	/comments/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Dar/Quitar like a comentario Toggle like en un comentario.

Path	/comments/{id}/like
Método	POST
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Agregar respuesta a comentario Añade una respuesta anidada a un comentario existente.

Path	/comments/{id}/replies
Método	POST
Parámetros	id en URL, content, <i>Token</i>
Retorno	id, message

Buscar posts (híbrida) Búsqueda combinada de texto completo (full-text) y semántica (embeddings).

Path	/community/search?q={query}
Método	GET
Parámetros	q (término de búsqueda), <i>Token en cabecera</i>
Retorno	Lista de posts con searchRank (puntuación de relevancia)

Buscar usuarios Búsqueda de usuarios por nombre, username o email usando trigramas.

Path	/community/search/users?q={query}&page={n}&limit={m}
Método	GET
Parámetros	q, page, limit (opcionales), <i>Token</i>
Retorno	Lista de usuarios con id, username, displayName, avatar, bio, followersCount, postsCount

Obtener posts recomendados Recomendaciones personalizadas basadas en interacciones previas del usuario.

Path	/community/recommended
Método	GET
Parámetros	<i>Token en cabecera</i>
Retorno	Lista de posts recomendados

Obtener feed personalizado Feed priorizado: posts de usuarios seguidos, con alta interacción o recientes.

Path	/community/personalized?limit={n}
Método	GET
Parámetros	limit (opcional, por defecto 10), <i>Token</i>
Retorno	Lista de posts ordenados por relevancia

Obtener posts en tendencia Posts más populares basados en likes y comentarios de los últimos días.

Path	/community/trending
Método	GET
Parámetros	<i>Token en cabecera</i>
Retorno	Lista de posts en tendencia

Obtener posts por etiqueta Filtra posts que contengan una etiqueta específica.

Path	/community/tags/{tag}
Método	GET
Parámetros	tag en URL, <i>Token en cabecera</i>
Retorno	Lista de posts con esa etiqueta

Seguir a un usuario El usuario autenticado comienza a seguir a otro usuario.

Path	/community/users/{userId}/follow
Método	POST
Parámetros	userId en URL, <i>Token en cabecera</i>
Retorno	message

Dejar de seguir a un usuario El usuario autenticado deja de seguir a otro usuario.

Path	/community/users/{userId}/follow
Método	DELETE
Parámetros	userId en URL, <i>Token en cabecera</i>
Retorno	message

Verificar si sigue a un usuario Comprueba si el usuario autenticado sigue a otro.

Path	/community/users/{userId}/is-following
Método	GET
Parámetros	userId en URL, <i>Token en cabecera</i>
Retorno	{isFollowing: true/false}

Obtener estadísticas de usuario Retorna contadores de seguidores, seguidos y posts de un usuario.

Path	/community/users/{username}/stats
Método	GET
Parámetros	username en URL, <i>Token opcional</i>
Retorno	followersCount, followingCount, postsCount

Obtener posts de un usuario Lista los últimos 50 posts de un usuario específico.

Path	/community/users/{userId}/posts
Método	GET
Parámetros	userId en URL, <i>Token en cabecera</i>
Retorno	Lista de posts del usuario

Registrar vista de post Registra que un usuario ha visto un post (para el feed personalizado).

Path	/community/posts/{id}/view
Método	POST
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera **Authorization: Bearer <token>**, excepto /health.*

8.4 Servicio de Deudas (debt)

8.4.1 Descripción General

El servicio **debt** gestiona préstamos y deudas del usuario, diferenciando entre dinero prestado (borrowed) y dinero prestado a terceros (lent). Permite registrar deudas con intereses fijos o variables, capitalización de intereses (compound), pagos parciales, y seguimiento del saldo restante. Cada deuda genera automáticamente una transacción asociada y actualiza el balance de la wallet correspondiente. Se integra con PostgreSQL (tablas **debts**, **debt_payments**, **variable_interests**), Redis para caché, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.4.2 Endpoints de la API REST

Obtener todas las deudas del usuario Retorna todas las deudas del usuario autenticado, incluyendo pagos e intereses variables asociados.

Path	/debts
Método	GET
Parámetros	<i>Token en cabecera Authorization</i>
Retorno	Lista de deudas con id , type , creditorName , originalAmount , remainingBalance , interestRate , interestType , compoundMonths , termMonths , monthlyPayment , startDate , nextDueDate , status , notes , realAmountToPay , realInterests , walletId , categoryId , createdAt , payments , variableInterests

Obtener deuda por ID Retorna una deuda específica del usuario autenticado.

Path	/debts/{id}
Método	GET
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	Objeto de la deuda con id , type , creditorName , originalAmount , remainingBalance , status , monthlyPayment

Crear deuda Registra una nueva deuda. Si es **borrowed** (dinero recibido), aumenta el balance de la wallet; si es **lent** (dinero prestado), lo disminuye. Crea automáticamente una transacción asociada.

Path	/debts
Método	POST
Parámetros	type ("borrowed"/"lent"), creditorName , walletId , categoryId , originalAmount , interestRate , interestType ("fixed"/"variable"), compoundMonths (0 = sin capitalización), termMonths , monthlyPayment , startDate , nextDueDate (opcional), notes (opcional), realAmountToPay (opcional), realInterests (opcional)
Retorno	id , message , type , creditorName , originalAmount , compoundMonths

Actualizar deuda Actualiza campos de una deuda existente. Si cambia `walletId` o `originalAmount`, ajusta automáticamente los balances de las wallets.

Path	/debts/{id}
Método	PUT
Parámetros	id en URL, campos a actualizar: <code>remainingBalance</code> , <code>status</code> , <code>nextDueDate</code> , <code>notes</code> , <code>originalAmount</code> , <code>interestRate</code> , <code>monthlyPayment</code> , <code>realAmountToPay</code> , <code>realInterests</code> , <code>compoundMonths</code> , <code>walletId</code> (todos opcionales), <i>Token</i>
Retorno	message, id

Eliminar deuda Elimina una deuda y sus pagos asociados. Reversa el efecto en el balance de la wallet y elimina la transacción asociada.

Path	/debts/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message, id

Registrar pago de deuda Registra un pago parcial o total de una deuda. Actualiza el saldo restante, ajusta el balance de la wallet asociada y crea una transacción automática.

Path	/debts/{id}/payments
Método	POST
Parámetros	debtId, amount, date, interestAmount, principalAmount, remainingBalance, notes (opcional)
Retorno	id, message, amount, remainingBalance

Eliminar pago de deuda Elimina un pago registrado. Reversa el efecto en el saldo de la deuda y en el balance de la wallet.

Path	/payments/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Registrar interés variable Agrega una tasa de interés variable para un mes específico de una deuda (solo para `interestType` = "variable").

Path	/interests
Método	POST
Parámetros	debtId, month, rate
Retorno	message

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera **Authorization: Bearer <token>**, excepto **/health**.*

8.5 Servicio de Metas (goal)

8.5.1 Descripción General

El servicio `goal` gestiona las metas de ahorro del usuario. Permite definir objetivos financieros con monto objetivo, fecha límite, prioridad y contexto. Cada meta puede recibir aportes (`add`) o retiros (`remove`) que actualizan el monto actual, generan transacciones automáticas y ajustan el balance de las wallets asociadas. Se integra con PostgreSQL (tablas `goals`, `goal_transactions`), Redis para caché, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.5.2 Endpoints de la API REST

Obtener todas las metas del usuario Retorna todas las metas del usuario autenticado, incluyendo sus transacciones asociadas.

Path	<code>/goals</code>
Método	GET
Parámetros	<i>Token en cabecera Authorization</i>
Retorno	Lista de metas con <code>id</code> , <code>name</code> , <code>targetAmount</code> , <code>currentAmount</code> , <code>targetDate</code> , <code>priority</code> , <code>status</code> , <code>context</code> , <code>purchaseCategoryId</code> , <code>completedAt</code> , <code>createdAt</code> , <code>transactions</code>

Obtener meta por ID Retorna una meta específica del usuario autenticado.

Path	<code>/goals/{id}</code>
Método	GET
Parámetros	<code>id</code> en URL, <i>Token en cabecera</i>
Retorno	Objeto de la meta con <code>id</code> , <code>name</code> , <code>targetAmount</code> , <code>currentAmount</code> , <code>status</code> , <code>priority</code> , <code>targetDate</code> , <code>context</code>

Crear meta Registra una nueva meta de ahorro.

Path	<code>/goals</code>
Método	POST
Parámetros	<code>name</code> , <code>targetAmount</code> , <code>targetDate</code> , <code>priority</code> ("high"/"medium"/"low", opcional, por defecto "medium"), <code>currentAmount</code> (opcional, por defecto 0), <code>context</code> (opcional), <code>purchaseCategoryId</code> (opcional)
Retorno	<code>id</code> , <code>message</code> , <code>name</code> , <code>targetAmount</code>

Actualizar meta Actualiza campos de una meta existente.

Path	<code>/goals/{id}</code>
Método	PUT
Parámetros	<code>id</code> en URL, campos a actualizar: <code>name</code> , <code>targetAmount</code> , <code>currentAmount</code> , <code>targetDate</code> , <code>priority</code> , <code>status</code> , <code>context</code> , <code>completedAt</code> , <code>purchaseCategoryId</code> (todos opcionales), <i>Token</i>
Retorno	<code>message</code> , <code>id</code>

Eliminar meta Elimina una meta y todas sus transacciones asociadas. Reversa el efecto en los balances de las wallets.

Path	/goals/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Agregar transacción a meta Registra un aporte (add) o retiro (remove) a una meta. Actualiza el monto actual, crea una transacción automática y ajusta el balance de la wallet.

Path	/goal-transactions
Método	POST
Parámetros	goalId, amount, type ("add"/"remove"), date, walletId, note (opcional)
Retorno	id, message

Eliminar transacción de meta Elimina una transacción de meta. Reversa el efecto en el monto actual de la meta y en el balance de la wallet.

Path	/goal-transactions/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera **Authorization: Bearer <token>**, excepto **/health**.*

8.6 Servicio de Hábitos (habit)

8.6.1 Descripción General

El servicio **habit** gestiona los hábitos del usuario, permitiendo el seguimiento de actividades diarias mediante checks (marcas de cumplimiento). Cada hábito almacena un streak actual (racha consecutiva de días cumplidos) y un best streak (racha máxima histórica). Los checks se almacenan en una columna JSONB, lo que permite flexibilidad en el seguimiento sin necesidad de tablas adicionales. Se integra con PostgreSQL (tabla **habits**), Redis para caché, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.6.2 Endpoints de la API REST

Obtener todos los hábitos del usuario Retorna todos los hábitos del usuario autenticado, incluyendo sus checks diarios.

Path	/habits
Método	GET
Parámetros	<i>Token en cabecera Authorization</i>
Retorno	Lista de hábitos con id, name, isActive, currentStreak, bestStreak, createdAt, checks (array con checkDate, completed, note)

Crear hábito Registra un nuevo hábito para el usuario autenticado.

Path	/habits
Método	POST
Parámetros	name (nombre del hábito)
Retorno	id, message

Actualizar hábito Actualiza los campos de un hábito existente.

Path	/habits/{id}
Método	PUT
Parámetros	id en URL, campos a actualizar: name, isActive, currentStreak, bestStreak (todos opcionales), Token
Retorno	message

Eliminar hábito Elimina un hábito y todos sus checks asociados.

Path	/habits/{id}
Método	DELETE
Parámetros	id en URL, Token en cabecera
Retorno	message

Registrar check de hábito Registra el cumplimiento (completado o no) de un hábito en una fecha específica. Actualiza automáticamente el currentStreak y bestStreak del hábito.

Path	/habits/{id}/check
Método	POST
Parámetros	id en URL, checkDate (YYYY-MM-DD), completed (boolean), note (opcional), Token
Retorno	message

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera **Authorization: Bearer <token>**, excepto /health.*

8.7 Servicio de Simulación (simulation)

8.7.1 Descripción General

El servicio `simulation` permite al usuario modelar transacciones recurrentes (ingresos o gastos) y calcular su impacto en diferentes períodos de tiempo (días, semanas, meses). Cada simulación almacena un monto, una categoría, un período, y las fechas de inicio y fin, junto con los valores calculados de días, semanas y meses que abarca. Se integra con PostgreSQL (tabla `simulation_transactions`), Redis para caché, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.7.2 Endpoints de la API REST

Obtener todas las simulaciones del usuario Retorna todas las simulaciones del usuario autenticado.

Path	/simulations
Método	GET
Parámetros	<i>Token en cabecera Authorization</i>
Retorno	Lista de simulaciones con id, amount, categoryId, categoryName, description, startDate, endDate, days, weeks, months, period, createdAt

Crear simulación Registra una nueva simulación de transacción recurrente.

Path	/simulations
Método	POST
Parámetros	amount, categoryId, startDate, endDate, days, weeks, months, period ("day"/"week"/"month"), description (opcional)
Retorno	id, message

Actualizar simulación Actualiza los campos de una simulación existente.

Path	/simulations/{id}
Método	PUT
Parámetros	id en URL, campos a actualizar: amount, categoryId, description, startDate, endDate, days, weeks, months, period (todos opcionales), <i>Token</i>
Retorno	message

Eliminar simulación Elimina una simulación existente.

Path	/simulations/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera **Authorization: Bearer <token>**, excepto **/health**.*

8.8 Servicio de Transacciones (transaction)

8.8.1 Descripción General

El servicio **transaction** gestiona todas las transacciones financieras del usuario (ingresos y gastos). Cada transacción está asociada a una wallet (de donde se debita o acredita el dinero), una categoría (que determina si es ingreso o gasto), y puede incluir etiquetas (tags) para organización adicional. Al crear, actualizar o eliminar una transacción, el servicio ajusta automáticamente el balance de la wallet correspondiente según el tipo de categoría (income = aumenta balance, expense = disminuye balance). Se integra con PostgreSQL (tabla **transactions**), Redis para caché, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.8.2 Endpoints de la API REST

Obtener transacciones del usuario Retorna todas las transacciones del usuario autenticado, paginadas.

Path	/transactions
Método	GET
Parámetros	limit (máx 100, por defecto 100), offset (por defecto 0), <i>Token en cabecera</i>
Retorno	Lista de transacciones con id, amount, description, date, walletId, categoryId, categoryName, tags, createdAt

Obtener transacción por ID Retorna una transacción específica del usuario autenticado.

Path	/transactions/{id}
Método	GET
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	Objeto completo de la transacción

Crear transacción Registra una nueva transacción. Actualiza automáticamente el balance de la wallet asociada según el tipo de categoría.

Path	/transactions
Método	POST
Parámetros	amount, date, walletId, categoryId, description (opcional), tags (array de strings, opcional)
Retorno	id, message

Actualizar transacción Actualiza los campos de una transacción existente. Ajusta los balances de las wallets afectadas (origen y destino) según los cambios.

Path	/transactions/{id}
Método	PUT
Parámetros	id en URL, campos a actualizar: amount, description, categoryId, walletId, date, tags (todos opcionales), <i>Token</i>
Retorno	message

Eliminar transacción Elimina una transacción existente. Reversa el efecto en el balance de la wallet asociada.

Path	/transactions/{id}
Método	DELETE
Parámetros	id en URL, <i>Token en cabecera</i>
Retorno	message

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera **Authorization: Bearer <token>**, excepto **/health**.*

8.9 Servicio de Carteras (wallet)

8.9.1 Descripción General

El servicio `wallet` gestiona las cuentas o carteras financieras del usuario (efectivo, cuenta bancaria, tarjeta de crédito, tarjeta de débito, otros). Cada wallet almacena un balance actual que se actualiza automáticamente cuando se crean, modifican o eliminan transacciones asociadas. Permite registrar información bancaria como nombre del banco y últimos 4 dígitos de la tarjeta, así como personalización con color e icono. Se integra con PostgreSQL (tabla `wallets`), Redis para caché, Kafka para eventos de auditoría, y Keycloak para autenticación.

8.9.2 Endpoints de la API REST

Obtener todas las carteras del usuario Retorna todas las carteras del usuario autenticado.

Path	/wallets
Método	GET
Parámetros	<i>Token en cabecera Authorization</i>
Retorno	Lista de carteras con <code>id</code> , <code>name</code> , <code>type</code> , <code>bankName</code> , <code>lastFourDigits</code> , <code>color</code> , <code>icon</code> , <code>currentBalance</code> , <code>isActive</code> , <code>createdAt</code>

Crear cartera Registra una nueva cartera financiera para el usuario autenticado.

Path	/wallets
Método	POST
Parámetros	<code>name</code> , <code>type</code> ("cash"/"bank_account"/"credit_card"/"debit_card"/"other"), <code>bankName</code> (opcional), <code>lastFourDigits</code> (opcional), <code>color</code> (opcional, por defecto "#6366F1"), <code>icon</code> (opcional, por defecto "Wallet"), <code>currentBalance</code> (opcional, por defecto 0)
Retorno	<code>id</code> , <code>message</code>

Actualizar cartera Actualiza los campos de una cartera existente.

Path	/wallets/{id}
Método	PUT
Parámetros	<code>id</code> en URL, campos a actualizar: <code>name</code> , <code>type</code> , <code>bankName</code> , <code>lastFourDigits</code> , <code>color</code> , <code>icon</code> , <code>currentBalance</code> , <code>isActive</code> (todos opcionales), <i>Token</i>
Retorno	<code>message</code>

Eliminar cartera Elimina una cartera existente.

Path	/wallets/{id}
Método	DELETE
Parámetros	<code>id</code> en URL, <i>Token en cabecera</i>
Retorno	<code>message</code>

Verificar salud del servicio Endpoint de health check.

Path	/health
Método	GET
Parámetros	-
Retorno	Estado del servicio y conectividad con Redis

*Nota: La URL base es <https://yung-accountant-back.duckdns.org>. Todos los endpoints requieren autenticación con token JWT en cabecera *Authorization: Bearer <token>*, excepto */health*.*

8.10 Frontend (React + TypeScript)

8.10.1 Descripción General

El frontend de Yung Accountant es una aplicación single-page (SPA) desarrollada en React 19 con TypeScript, Vite como bundler y TailwindCSS para estilos. Implementa una arquitectura basada en stores (Zustand) para gestión de estado, servicios con Axios para comunicación con la API, y componentes reutilizables con sistema de temas dinámicos (glassmorphism brutalista). La aplicación consume los microservicios a través de un API Gateway unificado en <https://yung-accountant-back.duckdns.org>.

8.10.2 Arquitectura del Frontend

Patrón de comunicación El frontend sigue un flujo unidireccional de datos:

1. **Componentes UI** (páginas y componentes) disparan acciones.
2. **Stores (Zustand)** contienen la lógica de negocio y el estado.
3. **Servicios (Axios)** realizan peticiones HTTP a la API.
4. **API Gateway** redirige a los microservicios correspondientes.
5. **Respuesta** actualiza los stores y refresca la UI.

Gestión de estado (Zustand) La aplicación utiliza múltiples stores especializados con persistencia parcial en localStorage:

- **useUserStore:** Autenticación, perfil de usuario, seguidores, caché de usuarios públicos.
- **useCategoryStore:** Categorías de ingresos/gastos (sistema + usuario).
- **useWalletStore:** Carteras y balances.
- **useTransactionStore:** Transacciones financieras.
- **useGoalStore:** Metas de ahorro y sus transacciones.
- **useDebtStore:** Deudas, préstamos y pagos.
- **useHabitStore:** Hábitos y rachas (streaks).
- **useCommunityStore:** Posts, comentarios, likes, feed personalizado.
- **useSimulationStore:** Simulaciones de transacciones recurrentes.

Cada store implementa:

- **Caché en memoria** con TTL de 5 minutos (evita refetch innecesario).
- **Persistencia selectiva** (tokens de autenticación) en localStorage.
- **Invalidación automática** tras operaciones de escritura (create/update/delete).
- **Optimistic updates** para acciones como likes (actualización inmediata + rollback en error).

Servicios y comunicación API Los servicios encapsulan todas las llamadas HTTP usando Axios con interceptores:

- **Interceptor de request:** Agrega automáticamente el token JWT (`Authorization: Bearer <token>`).
- **Interceptor de response:** Maneja errores 401 (token expirado) y dispara evento `auth:unauthorized`.
- **Bases URL:** Todas apuntan al mismo API Gateway (los paths diferencian el microservicio).

Sistema de temas (Themes) La aplicación implementa 6 temas diferentes según el rol del usuario:

- **estudiante-dark:** Azul profundo (noche de estudio, concentración).
- **estudiante-light:** Azul cristal (mañana productiva, claridad mental).
- **trabajador-dark:** Vidrio y mercurio (instrumental científico, botellas de vidrio).
- **trabajador-light:** Cristalería de laboratorio, plata esterlina.
- **ama-de-casa-dark:** Violeta cálido (atardecer hogareño, creatividad).
- **ama-de-casa-light:** Violeta nube (mañana en casa, calma).

El tema se aplica dinámicamente mediante atributo `data-theme` en el elemento raíz y variables CSS personalizadas.

8.10.3 Endpoints del Frontend (Rutas)

El frontend utiliza React Router DOM para la navegación:

Ruta	Descripción
/	Página de inicio (landing) o redirección a dashboard si autenticado
/login	Inicio de sesión
/register	Registro de nuevo usuario
/dashboard	Panel principal con resumen financiero
/transactions	Listado y gestión de transacciones
/wallets	Gestión de carteras/cuentas
/categories	Gestión de categorías
/goals	Gestión de metas de ahorro
/debts	Gestión de deudas y préstamos
/habits	Seguimiento de hábitos diarios
/community	Comunidad: posts, feed, búsqueda
/community/post/{id}	Vista detallada de un post
/simulation	Simulador de transacciones recurrentes
/profile/{username}	Perfil público de usuario
/settings	Configuración de cuenta
/help	Ayuda y preguntas frecuentes

8.10.4 Flujo de Autenticación

1. Usuario completa formulario de login/registro.
2. Store llama al servicio `authService.login()`.
3. Axios envía petición POST a `/auth/login`.
4. Backend retorna `accessToken` y `refreshToken`.
5. Store guarda tokens en `localStorage` y estado de Zustand.
6. Interceptor de Axios automáticamente añade token a todas las peticiones posteriores.
7. Al expirar el token (401), el interceptor intenta refrescar automáticamente.
8. Si el refresh falla, se dispara evento `auth:unauthorized` y se redirige a login.

8.10.5 Flujo de Búsqueda (Navbar)

1. Usuario ingresa texto en barra de búsqueda y selecciona modo (posts/users).
2. Al presionar Enter o botón de búsqueda, navega a `/community?search=query&mode=mode`.
3. Página `Community` lee parámetros de URL y ejecuta búsqueda correspondiente.
4. Store `useCommunityStore` llama al servicio apropiado.
5. Servicio hace petición a `/community/search` (posts) o `/community/search/users` (users).
6. Resultados se muestran en la página de comunidad.

8.10.6 Optimistic Updates (Ejemplo: Likes)

1. Usuario hace click en like de un post.
2. Store actualiza inmediatamente el estado local (`likesCount +1`, agrega `userId` a `likedBy`).
3. La UI se actualiza instantáneamente (sin esperar respuesta).
4. En segundo plano, se hace petición POST a `/community/posts/{id}/like`.
5. Si la petición falla, se revierte el cambio (refresca datos desde backend).
6. El backend invalida caché de Redis automáticamente.

8.10.7 Tecnologías y Librerías

- **React 19 + TypeScript:** Framework principal con tipado estático.
- **Vite:** Bundler rápido para desarrollo y producción.
- **TailwindCSS 4:** Utilidades CSS con sistema de diseño glassmorphism.
- **Zustand:** Gestión de estado ligera con persistencia.
- **Axios:** Cliente HTTP con interceptores para tokens.
- **React Router DOM:** Enrutamiento declarativo.
- **Chart.js + react-chartjs-2:** Visualización de datos (gráficos).
- **Lucide React:** Íconos SVG modernos.
- **date-fns:** Manipulación de fechas.
- **clsx:** Utilidad para clases condicionales.

8.10.8 Configuración de Entorno (Frontend)

- `VITE_API_URL`: URL base del API Gateway (por defecto `https://yung-accountant-back.duckdns.org`)
- `VITE_KEYCLOAK_URL`: URL de Keycloak (para redirecciones OAuth, si aplica)

8.10.9 Características Destacadas

- **Glassmorphism brutalista:** Componentes con efecto vidrio (`backdrop-filter`), bordes sutiles y sombras.
- **Temas por rol:** 6 combinaciones de color diferentes según el perfil del usuario.
- **Responsive design:** Adaptación completa a dispositivos móviles con menú lateral colapsable y modal de búsqueda.
- **Caché inteligente:** TTL de 5 minutos en stores, invalidación automática tras escritura.
- **Optimistic UI:** Actualizaciones inmediatas con rollback en caso de error.
- **Scroll personalizado:** Scrollbar estilizado con colores del tema actual.
- **Animaciones suaves:** Keyframes CSS con `fadeIn`, `scaleIn`, `slideIn`, `float`, etc.